

RESILIENCEOS

AI-Powered Intelligent Hospital Digital Twin for Infrastructure Resilience

Master Technical & Product Documentation

Document Version 1.0 · Status: Draft — Hackathon Prototype Specification

Team: 6-person student team (roles proposed, not finalized — see Part XVII)

Prepared for: College hackathon, Dehradun, India (independent event; not a Smart India Hackathon submission)

Document Status & How to Read This Document

This document is a single source of truth for the ResilienceOS concept, architecture, and build plan. It is written for four audiences at once: the development team (who will implement it), hackathon judges (who will evaluate it), the team itself during Q&A defense, and future contributors who may extend the prototype after the event.

Labeling convention. Throughout this document, every capability, number, or claim is tagged with one of five status labels, introduced formally in Part XIX and used consistently elsewhere:

- **[IMPLEMENTED]** — built and runnable in the hackathon prototype.
- **[PROPOSED / DESIGN]** — architected and intended for the hackathon build, not yet confirmed working end-to-end.
- **[FUTURE EXTENSION]** — explicitly out of scope for the hackathon; documented for roadmap purposes only.
- **[PROTOTYPE ASSUMPTION]** — a configurable parameter or simplification chosen for demonstration, not a validated real-world value.
- **[ILLUSTRATIVE EXAMPLE]** — a worked number used to explain a mechanism; not a claimed experimental result.

No number in this document should be read as a validated engineering, clinical, or experimental result unless it is explicitly cited to an external source with a reference.

Acronyms

Acronym	Meaning
ATS	Automatic Transfer Switch
BMS	Building Management System
CI	Critical Infrastructure
EES	Essential Electrical System (NFPA 99)
EPSS	Emergency Power Supply System (NFPA 110)
ER/ED	Emergency Department
HVAC	Heating, Ventilation, and Air Conditioning
ICU	Intensive Care Unit
IoT	Internet of Things
ML	Machine Learning
MVP	Minimum Viable Product
NIST	National Institute of Standards and Technology (USA)
NFPA	National Fire Protection Association (USA)
OT	Operating Theatre
SCADA	Supervisory Control and Data Acquisition
UPS	Uninterruptible Power Supply
WS	WebSocket
XAI	Explainable Artificial Intelligence

Part I — Executive Understanding

1. Executive Summary

Hospitals run on a web of interdependent infrastructure — grid power, backup generators, UPS and batteries, water, HVAC, and medical gas — that most monitoring systems still observe as separate, disconnected feeds. A building management system (BMS) can tell an operator that a transformer has failed. It is much less able to tell them, immediately and clearly, which hospital services that failure will threaten next, how much time is left before a threshold is crossed, and which of several possible responses best preserves critical care.

ResilienceOS is a hackathon prototype that closes this gap for a small, representative hospital model. It combines a dependency graph of infrastructure and services, a stateful simulation of how failures evolve over time, a small set of machine-learning components for anomaly and risk signals, and a 3D visualization layer, into one operational loop: observe the current state, understand what depends on what, predict how a failure may cascade, simulate candidate responses, compare their outcomes, explain the reasoning, and hand the decision to a human operator.

The system is explicitly a decision-support prototype, not an autonomous controller, and it is deliberately infrastructure-focused rather than clinical: it never claims to predict patient outcomes, and it uses no patient data of any kind.

2. One-Minute Explanation

Imagine a hospital's electricity fails. A normal monitoring dashboard shows a red light on the transformer. ResilienceOS instead shows the operator a chain of consequences: the transformer feeds an emergency bus, that bus powers the ICU, the ICU's HVAC and monitoring equipment depend on that power, and as the backup generator ramps up and the UPS batteries begin to discharge, the modeled risk to ICU service continuity rises over the following minutes. The operator can then test, in simulation, what happens if they shed non-critical loads, redistribute power, or prioritize specific services — and see, before acting, which strategy keeps the ICU and Emergency Department running longest with the least resource cost. ResilienceOS is the layer between "something failed" and "here is what to do about it."

3. The Problem

Hospital infrastructure systems are physically and operationally interdependent, but the tools operators use to monitor them are frequently siloed by system: an electrical BMS panel, a separate HVAC controller, a separate water/pump monitor, a separate medical-gas alarm panel. Each can report its own state accurately. None of them, by itself, answers the operational question that actually matters during an incident:

- What will this failure affect next, and how far will it spread?
- How much time do we have before a critical service is affected?
- Which of the available responses actually preserves the most critical care, and at what cost?

This is a known and well-studied class of problem in critical-infrastructure engineering generally — interdependent systems fail in ways that are difficult to reason about using single-system telemetry alone, because the consequences of a fault propagate along dependency paths that cross system boundaries. Rinaldi, Peerenboom, and Kelly's foundational 2001 analysis of infrastructure interdependency describes exactly this

dynamic: interconnected and mutually dependent infrastructures can produce cascading, escalating, and common-cause failures that are difficult to anticipate from any single system's point of view.[1] A hospital is a compact, safety-critical instance of this same class of interdependent system.

4. Why the Problem Matters

Hospitals are subject to some of the strictest backup-power regulation of any building type precisely because infrastructure failure has immediate safety consequences. In the United States, for example, NFPA 99 (the Health Care Facilities Code) requires a Type 1 Essential Electrical System for facilities with life-support-dependent patients, and NFPA 110 requires that emergency power be available within 10 seconds of a normal-power interruption for the systems it governs, with defined branches (Life Safety, Critical, and Equipment) and mandated periodic testing.[2] These rules exist because backup transitions are not instantaneous or risk-free: generator start-up has a delay, UPS/battery reserves are finite, and load must be managed carefully during the transition window. ResilienceOS is built around exactly this transition window — the minutes after a primary failure during which infrastructure state is changing rapidly and human decisions have the most leverage.

Prototype framing. ResilienceOS does not attempt to encode or verify compliance with NFPA 99/110 or any jurisdiction's actual code. These standards are cited here only to justify why the ten-to-twenty-minute post-failure window is operationally meaningful, and why hospital backup architectures are commonly organized into branches/buses of differing criticality — a structure ResilienceOS's dependency model deliberately mirrors.

5. Core Idea

ResilienceOS is a living digital model of critical hospital infrastructure that understands system dependencies, predicts how failures may cascade, simulates possible responses, and helps operators protect critical hospital services before disruption spreads.

Positioning statement:

Normal hospital systems tell you something has failed.
ResilienceOS tells you what that failure could affect next
and lets you test what to do about it before acting.

Central question the system is built to answer:

"If something fails, what will fail next —
and what can we do before it becomes critical?"

6. How ResilienceOS Works

Conceptually, the system is a pipeline, not a single black box:

```

Infrastructure Telemetry
↓
State Estimation
↓
Digital Twin Model
↓
Dependency Graph
↓
Cascade Analysis
↓

```

```

What-If Simulation
↓
Response Evaluation
↓
Explainable Options
↓
Human Operator
↓
Simulated Recovery

```

Each stage is a distinct, inspectable module (Part IV gives the full architecture). No single component — least of all a language model — is responsible for the whole pipeline; this separation is deliberate and is revisited in Part XIV (Hybrid Intelligence).

7. Core Innovation

The innovation claimed here is intentionally modest and specific. ResilienceOS does not claim to have invented digital twins, hospital monitoring, or cascading-failure analysis — all of these exist in prior research and commercial products (see Part IX, Existing System Comparison). The contribution is the integration of four capabilities into one interactive, demonstrable operator workflow, applied specifically to hospital-scale infrastructure:

1. Dependency-aware cascade analysis across electrical, backup-power, water, HVAC, and medical-gas systems within one graph model.
2. Interactive what-if intervention simulation that compares multiple response strategies on the same footing before any action is taken.
3. Service-level (not just asset-level) impact reporting, which keeps infrastructure state and hospital-service state as two distinct, explicitly related layers.
4. Concise, traceable explanation generated from the graph and simulation results rather than freely generated by a language model.

8. Key Features

Feature	What it does
Dependency Graph (Neo4j)	Represents infrastructure assets, hospital services, and the relationships between them, enabling traversal-based cascade queries.
Stateful IoT Simulator	Generates telemetry that evolves from modeled system state (not random noise), driven by a failure/backup state machine.
Cascade Engine	Propagates a failure through the dependency graph, applying capacity/threshold logic to determine downstream effects.
What-If Simulator	Runs six configurable response strategies against the same incident and reports comparable outcomes.
Resilience Index	A single 0–100 project-defined score summarizing modeled infrastructure/service health for a given state.
Explainability Layer	Produces short, graph-and-simulation-grounded causal explanations for "why is X at risk" and "why was Y recommended."
3D Digital Twin	Visualizes modeled hospital areas, infrastructure paths, and live state so a viewer can see the cascade, not just read about it.

Feature	What it does
Real-Time Dashboard	WebSocket-driven live view of telemetry, incidents, dependencies, and simulation results.

9. Target User

The primary persona for the prototype is a hospital infrastructure/facilities operator — someone responsible for electrical, mechanical, and utility systems, not clinical staff and not patients. This is a deliberate scope boundary: ResilienceOS supports infrastructure decisions, not clinical ones (see Part XV, Ethical/Safety Boundary). No additional user roles are introduced for the hackathon prototype beyond this single operator persona, to keep the demo focused.

10. Expected Impact

Within the scope of a hackathon prototype, the realistic and honestly stated impact is: a demonstrable proof of concept showing that dependency-aware cascade reasoning and intervention simulation can be built and visualized for a representative hospital infrastructure model, using synthetic data, in a way that a human operator could plausibly use as a decision aid after further engineering, integration with real telemetry, and validation. It is not a claim of measured reduction in downtime, cost, or risk in any real facility — no such measurement has been made.

Part II — Problem & Research

11. Problem Definition

Formally, the problem ResilienceOS addresses is: given a hospital infrastructure system composed of heterogeneous, interdependent subsystems (electrical, backup power, water, HVAC, medical gas) supporting a set of hospital services with differing criticality, provide an operator with (a) real-time state awareness at both the infrastructure and service level, (b) a prediction of how an observed or injected fault may propagate through the dependency structure over time, (c) the ability to simulate and compare candidate interventions before committing to one, and (d) a concise explanation of the reasoning behind any risk assessment or recommendation.

12. Background

Hospitals are unusual among commercial buildings in that infrastructure failure has a direct and immediate bearing on patient safety, which is why they are subject to dedicated codes (NFPA 99, 110, 101, 70 in the U.S. context) governing essential electrical systems, generator start times, and testing regimes.[2] These codes recognize, structurally, that hospital power systems are organized into branches or buses of different criticality — a life-safety branch, a critical branch, and an equipment branch, in the NFPA 99 framing — which is one of the reasons ResilienceOS's own criticality-tiered bus/service model (Part V, Part VIII) is not an arbitrary invention but a simplified reflection of how such systems are already commonly organized in practice.

13. Hospital Infrastructure Interdependency

The general theory of infrastructure interdependency is well established outside the hospital domain. Rinaldi, Peerenboom, and Kelly's widely cited 2001 IEEE Control Systems Magazine paper classifies interdependencies as physical, cyber, geographic, and logical, and describes how failures in one infrastructure can produce cascading, escalating, or common-cause failures in coupled infrastructures.[1] Subsequent work has extended this into formal dependency-graph and input-output models for multi-order (not just first-order) effects.[3] ResilienceOS applies this general framing at hospital scale: grid, transformer, generator, UPS, battery, pump, HVAC plant, and medical-gas source are treated as interdependent infrastructure nodes, and ICU, OT, Emergency, and General Ward as dependent service nodes.

14. Digital Twins

The term "digital twin" is used precisely in this document, not loosely. NIST's draft NISTIR 8356 characterizes a digital twin as a virtual representation of a physical or perceived real-world entity, concept, or notion, and stresses that its value comes from real-time, bidirectional connection across the entity's lifecycle rather than from visual fidelity alone.[4] The broader digital-twin literature similarly distinguishes a true digital twin — one that is continuously synchronized with live data from its physical counterpart — from a static or offline simulation model that merely looks similar; treating any 3D or simulated model as a "digital twin" regardless of that synchronization is considered by much of this literature to be an overly broad, largely marketing-driven use of the term.[5] ResilienceOS is transparent about where it sits on this spectrum: for the hackathon prototype, the "physical counterpart" is a synthetic, simulated hospital rather than a live building, so the system is best described as a prototype digital twin of a simulated hospital infrastructure model — the twin behaviors (state,

dependency-aware simulation, predicted future states) are real, but the telemetry source is not a physical building (see Part XI for the full definitional treatment).

In healthcare specifically, digital-twin research has grown rapidly since roughly 2020. A 2024 IEEE Access survey (Balasubramanyam et al.) reviews healthcare digital-twin literature from 2020–2023 and frames the technology primarily around personalized/clinical applications and smart-hospital operations.[6] A systematic literature review by Xames and Topcu (IEEE Access, 2024) maps research trends and realization challenges specifically for healthcare-system digital twins, and a Cambridge Institute for Manufacturing case study (Mukherjee et al., 2023) works through the practical challenge of identifying which hospital processes and variables are worth digitally twinning in the first place, using a histopathology department as a concrete example.[7] The consistent theme across this literature is that most current healthcare digital-twin work targets clinical, patient-flow, or facility-maintenance use cases rather than dependency-aware infrastructure-resilience reasoning — which is the specific gap ResilienceOS's positioning targets (see Section 29, Research Gap).

15. Existing Monitoring Systems

Conventional hospital Building Management Systems (BMS) and SCADA-style monitoring provide asset-level telemetry, threshold alarms, and historical trending. They are mature, reliable, and not being replaced or reinvented by this project. What they typically do not provide out of the box is: an explicit, queryable dependency graph connecting infrastructure assets to the hospital services they support; a simulation capability for testing candidate responses before acting; or a resilience score that aggregates multiple infrastructure and service dimensions into one comparable number across scenarios. Where a specific commercial BMS does provide some of these capabilities, ResilienceOS does not claim otherwise — see the Existing System Comparison in Part IX for a category-level, not product-level, comparison.

16. Existing Research

Beyond the interdependency and digital-twin literature already cited, three further research areas are directly relevant and are drawn on throughout this document: (a) critical-infrastructure resilience engineering, which studies how systems absorb, adapt to, and recover from disruption rather than only how they fail; (b) graph-based dependency modeling for cascading-failure analysis, which underlies the choice of a property graph database in Part X; and (c) lightweight anomaly-detection methods such as Isolation Forest, introduced by Liu, Ting, and Zhou in 2008, which isolates anomalies via random recursive partitioning rather than modeling normal behavior directly, giving it linear time complexity and making it practical for a resource-constrained hackathon build.[8]

17. Literature Review

Research Area	What is established	Limitations / open questions	How ResilienceOS relates
Infrastructure interdependency (Rinaldi et al., 2001; later input-output/graph extensions)	Interdependent infrastructures produce cascading, escalating, and common-cause failure modes that single-system	General framework is largely qualitative/analytical; building-scale, operator-facing tooling is not its focus.	Applies the same interdependency classification at hospital scale and operationalizes it as a queryable graph

Research Area	What is established	Limitations / open questions	How ResilienceOS relates
	monitoring cannot capture.[1][3]		rather than a static analysis.
Digital twins (NIST NISTIR 8356; general DT literature)	A digital twin requires continuous, real, bidirectional synchronization with a physical counterpart to be a true (not marketing) digital twin.[4][5]	Standards and terminology are still maturing; many published systems blur the twin/simulation line.	Explicitly labels itself a prototype digital twin of a simulated hospital, not a twin of a live building, per Part XI.
Healthcare digital twins (Balasubramanyam et al. 2024; Xames & Topcu 2024; Mukherjee et al. 2023)	Rapidly growing body of work, concentrated on clinical/personalized-medicine and facility-maintenance applications.[6][7]	Dependency-aware, cascade-focused infrastructure-resilience use cases are comparatively underrepresented.	Targets this underrepresented niche: infrastructure resilience and service continuity, explicitly patient-data-free.
Anomaly detection (Liu, Ting & Zhou, 2008 — Isolation Forest)	Isolation-based detection is linear-time, low-memory, and effective without needing to model "normal" behavior explicitly.[8]	Designed for general tabular anomaly detection, not purpose-built for physical infrastructure semantics or time-series causality.	Selected as a lightweight, implementable anomaly-scoring component over telemetry windows; not treated as the system's core reasoning engine (see Part XVI).

18. Research Gap / Opportunity

The gap this project targets is not the existence of digital twins, knowledge graphs, or anomaly detection individually — all are established techniques — but the comparative scarcity of small, demonstrable systems that combine dependency-graph cascade reasoning with interactive what-if intervention simulation and service-level (not just asset-level) impact reporting, specifically for hospital infrastructure resilience rather than clinical or maintenance use cases. Existing research demonstrates strong results in general infrastructure-interdependency modeling, healthcare digital twins, and anomaly detection separately; ResilienceOS's opportunity is to combine these ideas into one interactive, hospital-scale prototype focused on the observe → predict → simulate → decide loop.

19. ResilienceOS Positioning

Position statement (repeated verbatim from Part I for consistency): ResilienceOS is a resilience decision-support engine visualized through a living digital twin — not primarily a 3D hospital, not primarily a dashboard, and not an autonomous controller. The 3D environment is the interface layer; the intelligence is the dependency graph, deterministic rules, probabilistic risk estimation, cascade analysis, simulation, evaluation, and explanation working together (Part IV).

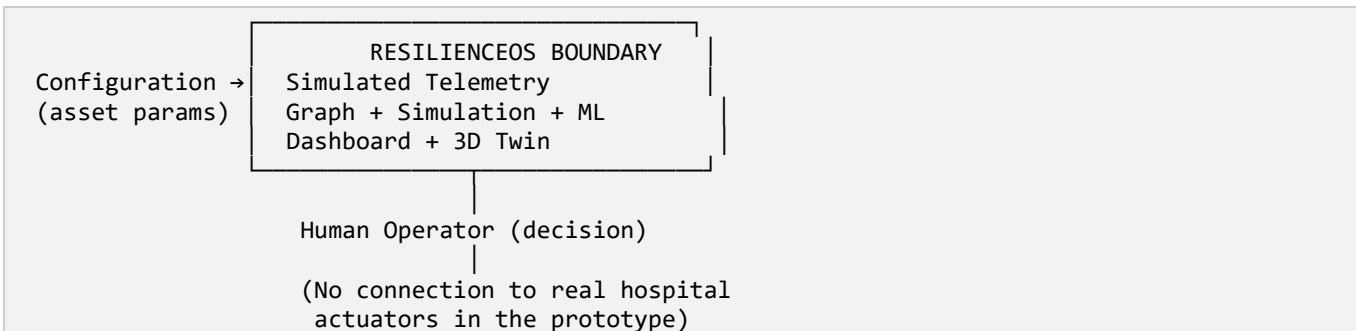
Part III — System Requirements

20. Scope

In scope for the hackathon prototype: a small, representative hospital model (ICU, Operating Theatre, Emergency Department, General Ward, Utility/Plant area, non-critical loads); simulated (not real) telemetry for electrical, backup-power, water, HVAC, and medical-gas systems; a dependency graph connecting infrastructure to services; a cascade-propagation engine; a what-if simulator supporting six response strategies; a project-defined Resilience Index; concise explainability; a real-time dashboard; and a 3D visualization of the modeled hospital and its infrastructure state.

Out of scope for the hackathon prototype: real hospital data or telemetry of any kind; patient data of any kind; integration with a live BMS/SCADA system; autonomous control of any real or simulated actuator beyond the simulation sandbox; clinical decision-making or diagnosis; production-grade security, authentication, and multi-tenant access control (a lightweight prototype-appropriate treatment is given in Part XV, with production requirements deferred to Future Extension).

21. System Boundary



Everything inside the boundary is simulated or configured by the team. No line in the architecture crosses into a real hospital's control systems for the hackathon build; a future BMS/SCADA integration path is discussed in Part XVI (Scalability) strictly as a labeled Future Extension.

22. Assumptions

- **[PROTOTYPE ASSUMPTION]** The hospital model represents a small subset of a real hospital's scale and complexity, chosen for demonstrability, not as a claim about typical hospital size.
- **[PROTOTYPE ASSUMPTION]** All numeric capacities, thresholds, delays, and criticality scores are configurable simulation parameters, not measured or certified engineering values.
- **[PROTOTYPE ASSUMPTION]** No two real hospitals necessarily share the same infrastructure topology; the modeled topology is one illustrative configuration, not a universal template (Part VII, Realism Rule).
- **[PROTOTYPE ASSUMPTION]** The six-person team has functioning web, backend, graph-database, and 3D-development skills sufficient to execute the roadmap in Part XVII within the hackathon timeframe.

23. Constraints

- Hackathon timeframe: the system must reach a working end-to-end demo loop (Part XX, Core MVP) within the competition's build window.
- Team size: six students, with proposed but not finalized role allocation (Part XVII).
- No access to real hospital infrastructure data (Part XII, Simulated IoT) — synthetic, state-driven telemetry is used instead.
- No claim of compliance with any jurisdiction's actual hospital electrical or safety code; NFPA and similar standards are used only as research context, not as a certification target.

24. Functional Requirements

ID	Requirement	Priority
FR-01	The system shall allow configuration of infrastructure assets, services, and their attributes (capacity, criticality, thresholds) without code changes, via a configuration model (Part XXIV).	Essential (MVP)
FR-02	The system shall ingest telemetry from the simulated IoT source at a configurable update interval.	Essential (MVP)
FR-03	The system shall push telemetry and state updates to connected dashboard clients in real time via WebSockets.	Essential (MVP)
FR-04	The system shall maintain a current state estimate (NORMAL / DEGRADED / AT RISK / FAILED) for every modeled infrastructure asset.	Essential (MVP)
FR-05	The system shall maintain a dependency graph of infrastructure and service nodes and their relationships, queryable via graph traversal.	Essential (MVP)
FR-06	The system shall support manual failure injection into any modeled infrastructure asset, with configurable severity and duration.	Essential (MVP)
FR-07	The system shall propagate an injected or detected failure through the dependency graph, applying threshold/capacity logic at each traversal step.	Essential (MVP)
FR-08	The system shall compute and report service-level impact (e.g. ICU Service Risk) as distinct from infrastructure-level state.	Essential (MVP)
FR-09	The system shall flag anomalous telemetry patterns using a lightweight statistical or ML method.	Valuable if time permits
FR-10	The system shall estimate a numeric risk score for services affected by an active incident.	Essential (MVP)
FR-11	Where sufficient state information exists (e.g. battery reserve and depletion rate), the system shall estimate time-to-threshold for the relevant resource.	Valuable if time permits
FR-12	The system shall support running What-If simulations for at least the six defined response strategies (Part X) against a given incident state.	Essential (MVP)
FR-13	The system shall present simulation results for multiple strategies in a directly comparable form (same metrics, same scale).	Essential (MVP)
FR-14	The system shall compute a Resilience Index (0–100) for any given modeled state.	Essential (MVP)
FR-15	The system shall generate a concise, graph/simulation-grounded explanation for a given risk assessment or recommended strategy.	Essential (MVP)
FR-16	The system shall synchronize the 3D visualization with current modeled infrastructure and service state.	Essential (MVP)

ID	Requirement	Priority
FR-17	The system shall present an incident timeline showing key state transitions over simulated time.	Valuable if time permits
FR-18	The system shall present response options to a human operator and require explicit operator action to "apply" a simulated strategy; it shall not auto-apply any strategy.	Essential (MVP)
FR-19	The system shall simulate recovery back toward a NORMAL state after a selected intervention is applied.	Essential (MVP)

25. Non-Functional Requirements

Category	Requirement
Performance	Dashboard telemetry updates should render within roughly one second of a simulated state change to preserve the feel of a live system during the demo.
Responsiveness	The 3D twin and dashboard views should remain interactive (no visible freeze) while a cascade or simulation is computing.
Scalability	The graph schema and simulation engine should not hard-code the number of assets/services, so the model can grow from one hospital wing to a larger configuration without a redesign (implementation performance at large scale is explicitly a Future Extension concern, not validated here).
Reliability	A failed WebSocket connection should not corrupt simulation state; the client should be able to reconnect and resynchronize.
Maintainability	Infrastructure and service definitions live in configuration/data, not hard-coded in application logic, so the hospital model can be edited without touching simulation code.
Modularity	Graph, simulation, ML, and explanation components are separate services/modules with defined interfaces (Part IV), so any one can be modified or replaced independently.
Observability	Key events (failure injection, cascade steps, simulation runs, resilience-index computation) are logged with enough detail to reconstruct what happened during a demo run.
Usability	A first-time viewer (a judge) should be able to understand the current incident and available responses without a verbal explanation, within the dashboard's information hierarchy (Part XII).
Extensibility	New infrastructure types (e.g., an additional utility system) can be added as new node/relationship types without breaking existing cascade logic.
Fault tolerance	The simulator should behave predictably (not crash or produce undefined state) under edge cases such as multiple simultaneous failures or missing telemetry (Part XXVI, Testing).
Reproducibility	A given scenario configuration and random seed (where randomness is used at all, e.g. anomaly-detector noise floors) should produce the same simulated outcome on repeated runs, to support reliable demoing.
Data integrity	Telemetry, state transitions, and simulation runs are persisted with enough structure (Part XXV) to reconstruct an incident after the fact.
Security (prototype-appropriate)	See Part XV: no patient data, no real actuation, and a concise treatment appropriate to a controlled hackathon environment rather than a production deployment.

26. User Journey

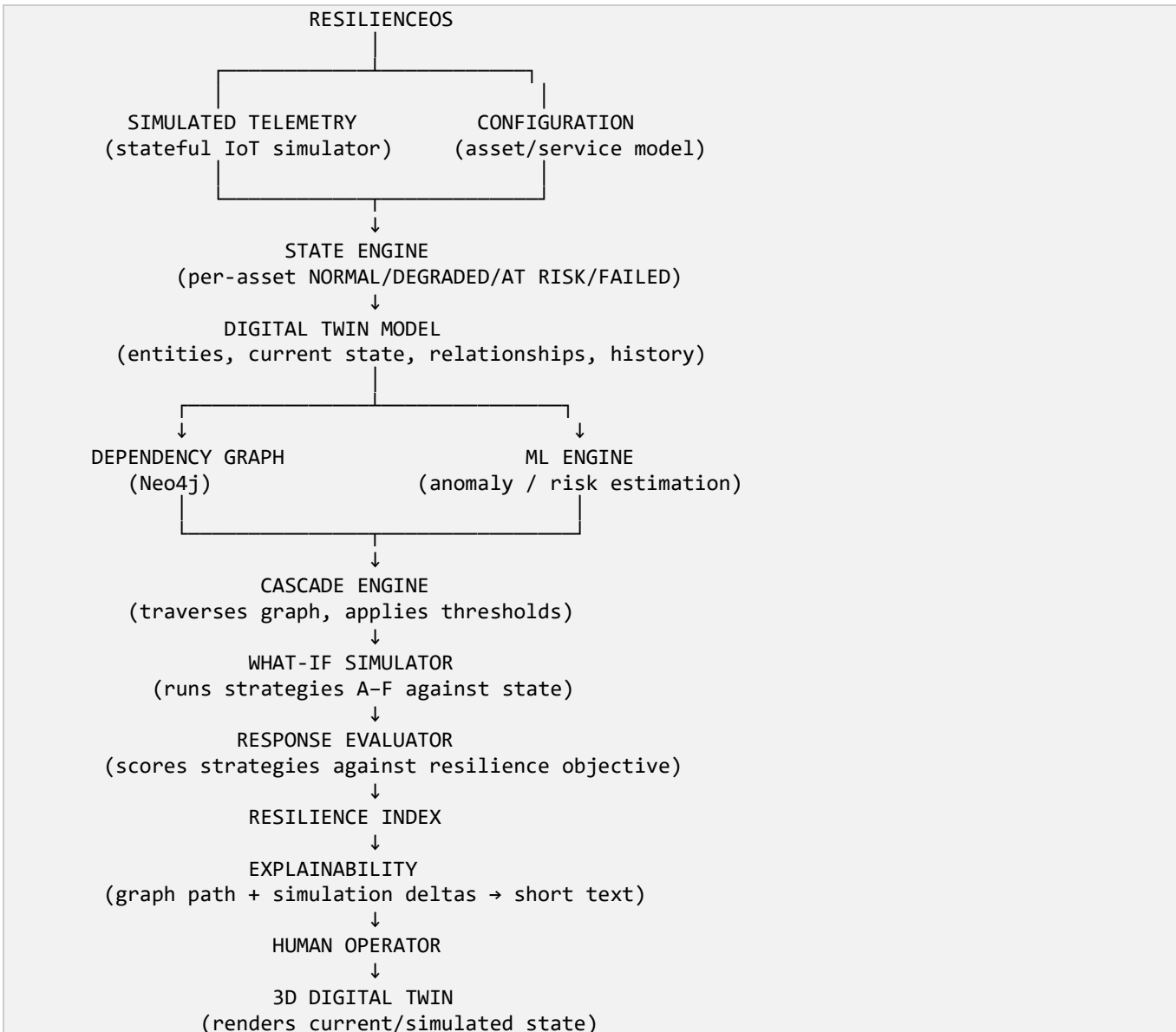
The full ten-step operator journey (view state → inspect services → inspect dependencies → trigger incident → watch cascade → inspect predicted impact → run what-if scenarios → compare strategies → inspect explanation → view simulated recovery) is specified in Part XII, Section 49, to avoid duplicating it here. No additional user roles are introduced.

27. Use Cases

Use Case	Primary Actor	Trigger	Outcome
UC-1: Monitor normal state	Operator	Operator opens dashboard	Operator sees current Resilience Index, infrastructure health, and service status with no active incident.
UC-2: Inspect dependencies	Operator	Operator opens Dependency View	Operator can trace which infrastructure assets support a given service, and vice versa.
UC-3: Inject a failure (demo/test)	Operator	Operator selects an incident type, severity, and duration	System begins propagating the failure and updating state in real time.
UC-4: Observe cascade	Operator	Active incident in progress	Operator sees the dependency chain and time-based progression of the cascade, plus rising service-risk indicators.
UC-5: Run what-if comparison	Operator	Operator opens What-If Lab during or after an incident	System runs the six strategies and returns comparable outcome metrics for each.
UC-6: Request explanation	Operator	Operator clicks "Why?" on a risk value or recommendation	System returns a short causal chain and/or bullet rationale derived from graph and simulation results.
UC-7: Apply a strategy (simulated)	Operator	Operator selects a strategy from the comparison view	System simulates the intervention's effect and shows the twin recovering toward NORMAL.

Part IV — System Architecture

28. Overall Architecture



28.1 Data flow

Telemetry flows one direction at each simulation tick: the IoT simulator advances the internal state machine for each asset (Part XXXVIII), publishes the resulting readings, the backend ingests and stores them, updates the State Engine's per-asset status, and — if a status change crosses a graph-relevant threshold — triggers the Cascade Engine to re-evaluate downstream nodes. Updated state is broadcast over WebSocket to all connected dashboard/3D clients.

28.2 Control flow

Control flow is operator-initiated at every decision point: failure injection, running a what-if scenario, requesting an explanation, and applying a simulated strategy are all explicit operator actions (FR-06, FR-12, FR-15, FR-18). The backend never autonomously injects a failure or applies a strategy on its own initiative.

28.3 Failure flow

A failure (injected or, in a future extension, ML-detected) sets the origin asset's state to FAILED or DEGRADED, which the Cascade Engine reads as the traversal seed. The engine walks outbound relationships (SUPPLIES, FEEDS, POWERS, BACKS_UP, COOLS, PROVIDES_WATER, PROVIDES_GAS, DEPENDS_ON) from that seed, at each hop checking whether the downstream node's available capacity, given the upstream reduction, remains above its configured threshold. Nodes that cross threshold have their own state updated and become new seeds for the next traversal layer (Part XVII gives full pseudocode).

28.4 Simulation flow

The What-If Simulator takes the current (or a specified historical) state plus a chosen strategy, applies the strategy's defined effect (e.g., "reduce non-critical load by configured percentage") to the relevant nodes, and re-runs the same cascade and risk-estimation logic forward across the configured time horizon, producing a comparable outcome record for each strategy (service continuity, backup margin, recovery time, resource cost, resilience index).

28.5 Recovery flow

Once a strategy is applied (simulated), the state engine transitions affected nodes along a DEGRADED → RECOVERING → NORMAL path at a rate determined by the strategy's configured recovery-time parameter, and the dashboard/3D twin animate this transition rather than jumping instantly back to NORMAL, so the demo can visibly show recovery rather than only failure.

29. Architecture Principles

5. Separation of reasoning types: deterministic graph rules, probabilistic ML, simulation, evaluation, and natural-language explanation are distinct modules with distinct responsibilities (Part XIV) — none is asked to do another's job.
6. Configuration over hard-coding: the hospital model is data, not code (Part XXIV), so the same engine can represent different hospital topologies.
7. Service state is derived, not assumed: infrastructure degradation does not automatically imply service failure; the relationship is computed via redundancy, backup, and threshold logic (Part XVIII).
8. Human-in-the-loop by construction: every consequential action (injecting a failure, applying a strategy) requires explicit operator input; nothing autonomously actuates.
9. Every claim is labeled: implemented vs. proposed vs. future, and validated-external-fact vs. project-assumption, throughout the document and, where feasible, in the UI itself (e.g., "Illustrative example" badges).

30. Data Layer

Two persistent stores are used for two different kinds of data, deliberately avoiding duplication (full schema in Part XXV–XXVI):

Store	Holds	Why
PostgreSQL (relational)	Telemetry time series, scenario/simulation run records, resilience-index history, alerts, configuration metadata	Well-suited to structured, tabular, time-ordered data with strong consistency needs.
Neo4j (graph)	Infrastructure assets, hospital services, and the typed relationships between them	Dependency traversal ("what does this failure reach, and through what path") is a graph-native query pattern; expressing it in a relational join-heavy schema would be considerably more complex to write and to reason about (Part X).

31. State Engine

The State Engine owns the authoritative current status of every modeled asset and service. It exposes a small state machine (Part XXXVIII) — NORMAL, DEGRADED, AT RISK, FAILED for infrastructure; and a parallel but distinct set of service-level statuses derived from infrastructure state plus dependency/redundancy logic (Part XVIII) — and is the single place other modules read "what is true right now" from.

32. Digital Twin

The Digital Twin layer composes the State Engine's current values with the graph's structure and a rolling history buffer, producing the object the 3D visualization and dashboard actually render. It is defined more formally, including its relationship to NIST's digital-twin definition, in Part XI.

33. Knowledge Graph

Neo4j stores nodes for infrastructure assets and hospital services and typed relationships between them (Part X, XXXVI). The Cascade Engine issues traversal queries (Cypher) against this graph rather than maintaining its own separate adjacency structure, so the graph is the single source of truth for "what depends on what."

34. ML Layer

Two lightweight ML components are scoped for the prototype (Part XVI): an anomaly-detection component over rolling telemetry windows, and a risk-estimation component that converts current state and trend into a probability-like risk score. Both are auxiliary to, not a replacement for, the deterministic graph/threshold logic — see Part XIV for the explicit division of responsibility.

35. Cascade Engine

Described in full in Part XVII. In architectural terms, it is a stateless service that takes a graph snapshot plus a set of origin failures and returns an ordered list of downstream state changes with the traversal path that produced each one — this path is exactly what the Explainability layer later renders as a causal chain.

36. Simulation Engine

Described in full in Part XXXVIII. Architecturally, it re-uses the Cascade Engine and State Engine logic, but runs them forward across a simulated time horizon under a hypothetical intervention rather than only reacting to the live state.

37. Response Evaluation

Takes the multiple simulated outcomes produced by the Simulation Engine and scores each against the resilience objective function defined in Part XLV, producing the ranked (but not auto-selected) comparison shown in the What-If Lab.

38. Explainability

A small, templated natural-language layer (Part XLIV) that converts a graph traversal path or a simulation delta into 3–6 lines of plain-language explanation. It does not free-generate causal claims; every sentence it produces must be traceable to a specific graph edge, threshold comparison, or simulation output value.

39. Real-Time Layer

A FastAPI backend exposes both a REST API (Part XXXVII) for on-demand queries/actions and WebSocket endpoints (Part XIII, XLVIII) for streaming telemetry, twin-state, and incident updates to connected clients.

40. Frontend

A React single-page application renders the dashboard views (Part XXVIII, L) and hosts the Three.js/React-Three-Fiber 3D twin (Part XXVII) as an embedded panel, so the 3D visualization and the 2D dashboard share the same live state stream rather than being two separately synchronized systems.

41. Backend

FastAPI (Python) hosts the REST/WebSocket API, orchestrates the simulator, cascade engine, ML components, and database access. Python is used throughout the backend and simulation/ML stack to keep the number of languages the six-person team must manage to two (Python, TypeScript/JavaScript for the frontend).

42. Database

See Part XXV (PostgreSQL conceptual schema) and Part XXVI (Neo4j graph schema) for full detail.

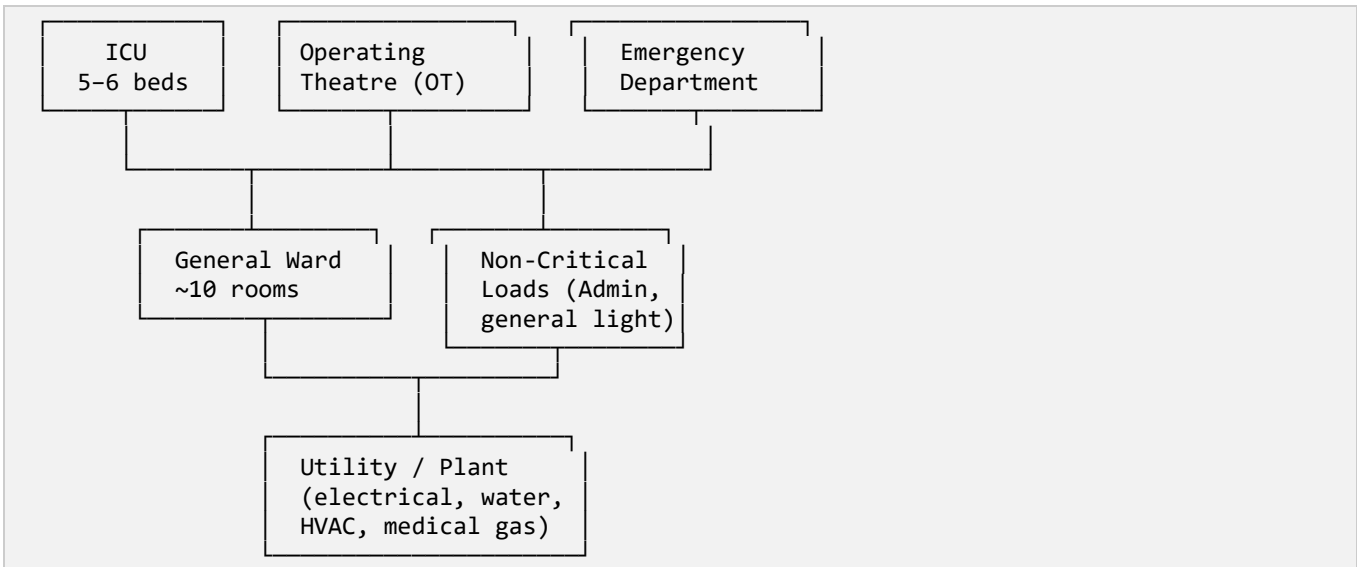
43. 3D Layer

Three.js via React Three Fiber renders a medium-detail hospital model (Part XXVII) whose materials/animations are driven by the same state objects the 2D dashboard consumes, so the two views cannot visually disagree about current state.

Part V — Hospital Model

The prototype represents a small but sufficiently complete hospital structure — not an entire real hospital — chosen to be large enough to create meaningful, multi-hop dependencies and small enough for six students to fully model, populate, and demo within a hackathon timeframe.

54. Hospital Layout



55. ICU

Scale (prototype assumption): 5–6 beds.

Modeled dependencies: electricity (via the emergency/critical bus), backup power (generator and, where applicable, UPS), HVAC, medical gas, water, and critical monitoring/equipment loads. Exact clinical or engineering requirements for ICU power/HVAC/gas provisioning are not invented; where the model requires a specific number (e.g., minimum acceptable temperature band, minimum gas pressure), it is marked a configurable prototype parameter, not a cited clinical standard.

56. Operating Theatre

Modeled dependencies: electrical demand (typically higher and steadier than a ward bed during active use), HVAC (theatres commonly require tighter environmental control than general wards), medical gas, and critical equipment loads. Criticality is configured equal to ICU and Emergency in the baseline scenario (Section 8), reflecting that an in-progress procedure has essentially zero tolerance for infrastructure interruption — this is a modeling choice, not a cited clinical standard.

57. Emergency Department

Modeled dependencies: critical electrical demand, HVAC, and general medical infrastructure dependencies (equipment power, some gas and water use). Criticality is configured at the same top tier as ICU and OT,

reflecting that Emergency Department load is unpredictable and cannot be scheduled around, unlike some General Ward or Administration functions.

58. General Ward

Scale (prototype assumption): ~10 rooms.

Modeled as a lower-criticality service relative to ICU/Emergency/OT (Section 8). This is a configurable prototype parameter, not a claim that general wards are universally less clinically important — it reflects that, in most backup-power architectures, general ward loads are more commonly shed or degraded before ICU/OT/Emergency loads during a capacity shortfall, which is exactly the asymmetry the demo is built to show.

59. Utility / Plant Area

Represents the supporting infrastructure systems: electrical supply and transformer/distribution, generator, UPS, batteries, water storage and pumps, HVAC plant, medical-gas system, and other supporting infrastructure needed to make the dependency model meaningful (Part VI).

60. Infrastructure Systems

At minimum, the model includes the four systems detailed in Part VI/VII/XXXIX–XLII: electrical (grid → transformer → main distribution → critical/non-critical buses), backup power (generator, UPS, battery, transfer/switching behavior), water (source → storage → pump → distribution), HVAC (plant → distribution → rooms/services), and medical gas (source → storage/regulation → distribution → services). Additional infrastructure components are added only when they create a meaningful new dependency or materially improve the demonstration — not to make the architecture diagram appear larger (Section 6/67, Anti-Buzzword Audit).

61. Criticality Model

Each modeled service carries a configurable criticality score on a 1–5 scale, used by the cascade engine, the response evaluator, and the Resilience Index to weight service-continuity outcomes. The baseline configuration (Part VIII) sets ICU, Emergency, and OT to criticality 5; General Ward to criticality 3; and Administration/non-critical loads to criticality 1. These are prototype configuration values, not universal clinical standards, and are expected to be adjusted per demo scenario or per hospital configuration.

62. Service Dependencies

Every service node declares its dependency edges explicitly rather than inheriting them implicitly. For example, the ICU service node requires electricity, HVAC, medical gas, and water as separate, independently trackable dependency edges, each with its own dependency-strength and failure-effect attributes (Part IX). This explicit declaration is what allows the cascade engine to reason about partial degradation (e.g., HVAC degraded but electricity intact) rather than treating a service as a single monolithic on/off unit.

Part VI — Knowledge Graph

63. Graph Model

ResilienceOS represents infrastructure and services as a labeled property graph, implemented in Neo4j. A property graph is a natural fit here because the central operational question — "if this node fails, what does it reach, and through what path?" — is a graph traversal, not a tabular join. Modeling the same structure relationally would require either a fixed-depth set of join tables (brittle as the topology changes) or a recursive common-table-expression pattern that most relational engines support less naturally and less efficiently than a graph engine's native traversal operators.

64. Nodes

Node label	Examples	Key properties
Infrastructure	Grid, Transformer, Generator, UPS, Battery, Pump, HVACPlant, MedicalGasSystem, Bus (Emergency/Main/Non-critical)	id, name, type, capacity, current_load, health_state, redundancy, threshold, location
Service	ICU, OperatingTheatre, EmergencyDepartment, GeneralWard, Administration	id, name, criticality, min_service_capacity, acceptable_degradation, backup_priority

65. Relationships

Relationship	Direction	Meaning
SUPPLIES	Infrastructure → Infrastructure	Upstream asset provides raw capacity to a downstream asset (e.g., Grid SUPPLIES Transformer).
FEEDS	Infrastructure → Infrastructure/Bus	Distribution-level provision (e.g., Transformer FEEDS Emergency Bus).
POWERS	Bus → Service	Electrical provision to a hospital service.
BACKS_UP	Generator/UPS/Battery → Bus	Backup-path relationship, activated under primary-supply failure.
DEPENDS_ON	Service → Infrastructure (generic)	Generic dependency where a more specific relationship type does not apply.
COOLS	HVACPlant → Service/Room	HVAC dependency.
PROVIDES_WATER	Pump/Storage → Service	Water dependency.
PROVIDES_GAS	MedicalGasSystem → Service	Medical-gas dependency.
SUPPORTS / AFFECTS	Generic	Used sparingly for relationships not covered by the above, to avoid an ever-growing ad hoc relationship taxonomy.

66. Properties

Relationship properties carry the quantitative meaning cascade logic depends on: `dependency_strength` (how much of the downstream node's function this edge accounts for, 0–1), `threshold` (the point below which the downstream node is considered affected), `failure_propagation_rule` (a reference to which threshold/capacity function to apply), `recovery_behavior`, and `priority` (used when multiple backup paths compete for limited capacity).

67. Graph Construction

The graph is built at startup from the configuration model (Part XXIV): each configured asset and service becomes a node; each configured dependency becomes a relationship with the properties above. This means the same code path that builds the demo hospital can build a different hospital configuration purely by changing configuration data — no cascade-engine code change is required to represent, for instance, an additional ward or an alternate backup topology.

68. Traversal

Cascade propagation is implemented as a breadth-first traversal seeded at the failed node(s), following outbound relationships layer by layer, evaluating threshold conditions at each hop before continuing to the next layer. Below is a representative Cypher pattern for finding everything reachable from a failed transformer:

```
MATCH path = (t:Infrastructure {id: 'T1'})
  -[:FEEDS|SUPPLIES|POWERS|COOLS|PROVIDES_WATER|PROVIDES_GAS*1..5]->(downstream)
RETURN path, downstream
ORDER BY length(path)
```

This query returns candidate downstream nodes and the path used to reach them; the Cascade Engine then applies threshold/capacity logic in application code to decide which of these candidates are actually affected, rather than assuming that reachability alone implies impact (Part XVII, XVIII). No query execution results are fabricated here — this is a representative pattern, not a captured output.

69. Failure Propagation

Failure propagation strategy and criticality propagation are specified in full in Part XVII (Failure Propagation Engine) and Part XXXVIII (Simulation Model); this section covers only how the graph supports that logic. Two design choices matter: first, traversal depth is capped (illustratively 5 hops) to keep propagation queries bounded and demo-latency predictable; second, each traversed edge's `dependency_strength` and the downstream node's current redundancy are read from the graph and combined multiplicatively (Part XLV) rather than treating any single failed upstream node as automatically failing everything downstream — this is what implements the Section 18 principle that infrastructure state and service state are distinct.

70. Explainability

Because the cascade result already carries the traversal path that produced it, the Explainability layer (Part XLIV) does not need to separately "figure out" why a node is at risk — it renders the path Neo4j already returned, in plain language. This is the central reason the graph representation was chosen over an opaque scoring model: the explanation is a byproduct of the computation, not a separate justification generated after the fact.

Why Neo4j specifically (anticipated judge question)

A relational database can express fixed-depth dependencies via explicit join tables, but hospital infrastructure dependency chains are variable-depth (grid → transformer → bus → service is four hops; a redundant path might be six), and expressing variable-depth traversal in SQL requires recursive CTEs that are both harder to write correctly and, in most engines, less efficient than a native graph traversal for this access pattern. Neo4j is not selected because "graph database" sounds advanced — the Anti-Buzzword Audit in Part XIX examines this choice explicitly — it is selected because the central operation the system performs, repeatedly, is exactly the operation Neo4j is built for.

Part VII — IoT & State

71. Telemetry Model

Telemetry values modeled: electrical load, power availability, generator state and load, UPS load, battery reserve, water pressure and pump state, HVAC temperature and load, medical-gas pressure, and per-asset health indicators. Every value is attached to a specific graph node so that a telemetry update can be mapped directly to a State Engine transition for that node.

72. Synthetic Data Generation

Why synthetic: No real hospital infrastructure telemetry dataset is available to the team, and real hospital telemetry — even if it were accessible — is sensitive, heterogeneous across facilities, and inappropriate to fabricate as if it were genuine. The prototype therefore uses a configurable, stateful synthetic telemetry simulator (Section 12) rather than either claiming access to real data it does not have, or presenting random values as if they were realistic sensor behavior.

What synthetic does NOT mean here: It does not mean `random.uniform()` noise with no relationship to system state. A telemetry value in ResilienceOS is always a function of the current simulated state machine (Part XXXVIII), not an independent random draw. For example, UPS load is computed from the currently connected critical-load total and the generator's ramp-up progress, not sampled independently of them.

73. Stateful Simulator

```

Grid failure
  ↓
Generator activation delay (configurable, e.g. 10s per NFPA 110
  Level 1 EPSS practice for reference context only – not a claimed
  compliance target)
  ↓
Generator load changes (ramps toward connected critical load)
  ↓
UPS response (bridges the gap during generator ramp-up)
  ↓
Battery reserve changes (depletes while UPS is the active source)
  ↓
Threshold crossing (battery reserve or generator margin below
  configured threshold)
  ↓
Service risk changes (computed via Part XVIII service-impact logic)

```

The generator start-time reference above cites NFPA 110's widely documented 10-second requirement for the emergency systems it governs purely as real-world context motivating why a short activation delay is modeled at all; the prototype's actual configured delay is a tunable parameter, not a compliance claim.[2]

74. Event Model

Two event categories drive the simulator: exogenous events (operator-injected failures, Part XXXIX's `FailureInjection` interface) and endogenous events (state-machine-triggered transitions, such as "battery reserve crossed 20% → emit `LOW_RESERVE` event"). Both event types are placed on the same internal event queue and

processed in simulated-time order, which is what allows the What-If Simulator to replay the same event model forward under a hypothetical intervention (Part XXII).

75. Real-Time Updates

Real-time communication uses WebSockets rather than polling, so that dashboard and 3D-twin clients receive state changes as they occur instead of on a fixed poll interval. Conceptual flow:

```

IoT Simulator
  ↓
FastAPI Backend
  ↓
WebSocket
  ↓
Dashboard
  ↓
3D Digital Twin
  
```

Illustrative example (not a claimed measured result):

```

Power:      420 → 400 → 360 → 300 kW
Generator:  OFF → STARTING → ON
UPS:        20% → 35% → 51%
Service Risk: 2% → 5% → 18% → 37%
  
```

76. State Transitions

Every infrastructure node moves through the same four-state machine — NORMAL, DEGRADED, AT RISK, FAILED — driven by comparing current telemetry against configured thresholds (Part XXXVIII gives the full transition diagram and guard conditions). Service nodes use a parallel but distinct status vocabulary (Part XVIII) precisely because service impact is not a direct copy of infrastructure state.

Part VIII — AI/ML

The AI/ML scope for this project is deliberately narrow. Machine learning is used where it adds a capability the deterministic graph/rules layer cannot provide (detecting a pattern that was not explicitly configured as a threshold, or scoring uncertainty), and nowhere else. Part XIV explains the hybrid-intelligence division of labor this scope follows.

77. Anomaly Detection

Method: Isolation Forest, introduced by Liu, Ting, and Zhou (2008), is the primary candidate method.[8] It isolates anomalies via recursive random partitioning rather than modeling "normal" behavior directly; anomalous points require fewer partitioning splits to isolate, which the algorithm converts into an anomaly score. Its linear time complexity and low memory footprint make it practical to run on a rolling telemetry window without a heavyweight training pipeline — a good fit for a hackathon-scale implementation.

Rolling statistical thresholds (moving average plus a robust z-score/median-absolute-deviation band) are kept as a fallback/complementary method: they are simpler to implement, explain, and validate under time pressure, and can serve as the anomaly signal if the Isolation Forest integration is not completed in time (Part XIX, MVP prioritization).

What this module is not: It is not a clinical anomaly detector and does not process patient data. It flags telemetry patterns (e.g., a battery discharging faster than its configured depletion curve) as statistically unusual, nothing more.

78. Failure-Risk Estimation

Risk estimation converts current infrastructure state, trend, and remaining margin into a 0–1 (displayed as a percentage) risk score for a given service, used both directly in the dashboard and as an input to the Resilience Index (Part XLVI). The baseline approach is a transparent, deterministic function of graph-derived features (remaining capacity margin, dependency-strength-weighted upstream degradation, and time-to-threshold where available) rather than an opaque trained classifier — this keeps the number traceable to specific causes, which the Explainability layer depends on.

79. Time-to-Threshold Estimation

Where enough state information exists (a current reserve level and an observed or configured depletion rate), the system estimates time remaining before a threshold is crossed:

Current reserve
÷
depletion rate
=
estimated time to threshold

This is a simple linear projection, not a validated predictive model; it is presented in the UI as an estimate, and the assumption of a constant depletion rate is stated explicitly wherever the estimate is shown.

80. ML Pipeline

```

Simulated telemetry stream
↓
Feature extraction (rolling window: mean, trend,
  rate of change, margin-to-threshold)
↓
Anomaly scoring (Isolation Forest / statistical fallback)
↓
Risk feature combination (deterministic formula, Part XLV)
↓
Risk score + anomaly flag → State Engine / dashboard

```

81. Feature Engineering

Candidate features per asset per rolling window: current value, mean, standard deviation, first difference (rate of change), distance to configured threshold (normalized), and time since last state transition. These are computed from simulator output, not learned from an external dataset, since none exists (Section 43).

82. Training

Where an actual model (e.g., Isolation Forest) is trained, it is trained on synthetic telemetry generated by running the simulator across a range of configured scenarios, deliberately including both normal-operation and induced-failure runs so the model sees examples of both classes of behavior it will need to distinguish. This training data is explicitly synthetic-only and is documented as such wherever training methodology is discussed — it is never described as, or substituted for, real-world validation data.

83. Validation

Because there is no real hospital dataset, formal validation against real-world ground truth is not possible for this prototype. Internal validation (does the anomaly detector flag the induced-failure runs and not the normal-operation runs, in the synthetic test set) is planned; this is clearly a synthetic self-consistency check, not external validation.

84. Evaluation

Metrics status: To be evaluated after implementation. No accuracy, precision, recall, F1, or ROC-AUC figures are reported in this document because none has yet been measured; reporting placeholder or assumed figures here would violate the No-Fabrication Rule (Part XXI). Once a working anomaly detector exists, the proposed evaluation methodology is to hold out a subset of synthetic scenarios (including edge cases such as multi-failure runs, Part XXXIX) and report standard classification metrics on that held-out synthetic set, explicitly labeled as synthetic-only results.

85. Explainable AI

Where SHAP, LIME, or permutation importance would be considered: they would be applicable to the risk-estimation model if and only if that model is a trained black-box classifier rather than the transparent deterministic formula described in Section 78. Given the current design (a formula over named, already-meaningful features), applying a post-hoc feature-attribution method would add complexity without adding explanatory value the formula does not already provide directly — so, per the Anti-Buzzword Audit (Part XIX),

SHAP/LIME are not included in the MVP. If a future version replaces the risk formula with a trained model whose behavior is not already transparent, SHAP-style attribution would become directly relevant and should be revisited then.

86. Limitations (AI/ML)

- All training and internal-validation data is synthetic; no claim of real-world predictive accuracy is made.
- The Isolation Forest / statistical anomaly detector has not yet been benchmarked against a labeled ground truth of real hospital anomalies, because none exists for this project.
- The linear time-to-threshold projection assumes a constant depletion/degradation rate, which real systems will not always exhibit.
- Risk scores are modeled infrastructure/service-risk indicators, not clinical or patient-risk probabilities (Section 19).

Part IX — Failure & Cascade Simulation

87. Failure Model

A failure is modeled as a reduction (partial or total) in an infrastructure node's available capacity or health state, with a configurable severity and duration. Failures are not assumed to always propagate to every downstream node; propagation is conditional on the configured dependency strength, redundancy, and threshold at each hop (Part VII, Realism Rule).

88. Cascade Engine

Full behavior specified in Part XVII; this section positions it within the failure/simulation family of modules. The engine performs ten steps for any incoming failure/degradation event: receive the event, update the origin node's state, traverse downstream dependencies, evaluate capacity/threshold conditions at each hop, propagate state changes where thresholds are crossed, identify affected infrastructure, identify affected services, calculate service-level impact, estimate risk/time-to-threshold where applicable, and generate the explainable dependency chain that becomes the traversal path used by Explainability (Part VI, Section 70).

89. Power Failure

The hero scenario (Part XX). Modeled chain, illustrative:

```

Transformer Failure
  ↓
Electrical Supply Reduced
  ↓
Emergency Bus Stress
  ↓
Backup Capacity Activated
  ↓
Generator Load Increases
  ↓
Backup Margin Decreases
  ↓
Potential HVAC Degradation
  ↓
OT / ICU Service Risk Increases
  
```

Full component-level detail (grid, transformer, main/emergency bus, generator, UPS, battery, critical/non-critical loads) is given in Part XXXIX.

90. Generator Failure

Modeled as the generator asset itself failing to start or dropping output after starting — a second-order failure on top of, or instead of, a primary grid failure. Because the graph already models BACKS_UP relationships explicitly, a generator failure is handled by the same traversal logic as any other node failure; no special-cased code path is needed, which is one practical benefit of the graph-native design (Part VI).

91. UPS/Battery Failure

Modeled as reduced UPS capacity or accelerated battery depletion. This scenario is particularly useful for exercising the Time-to-Threshold estimator (Part VIII, Section 79), since battery reserve is the clearest example of a resource with an observable current level and a computable depletion rate.

92. Water Failure

Modeled chain: water source → storage → pump → distribution → dependent services. State variables: pressure, pump state, storage level. Full detail in Part XL.

93. HVAC Failure

Modeled chain: HVAC plant → distribution → room/service dependencies. State variables: temperature, capacity, degradation. Full detail in Part XLI. No clinical claims are made about specific temperature thresholds' medical significance; thresholds are configurable prototype parameters.

94. Medical Gas Failure

Modeled chain: gas source → pressure/regulation → distribution → dependent services. Full detail in Part XLII. No treatment or clinical-outcome claims are made; only pressure/availability state is modeled.

95. Multi-Failure Scenarios

The cascade engine can accept more than one simultaneous origin failure (e.g., transformer degradation plus generator degradation, or grid failure plus HVAC degradation), because the traversal and threshold logic operate per-node regardless of how many origin events exist; the only addition needed is seeding the initial traversal queue with multiple origins instead of one. The MVP does not need to specially engineer for this case, but it should not break under it (Part XXVI, Edge Cases).

96. Recovery

After an intervention is applied (simulated) or a failure duration expires, affected nodes transition through a RECOVERING intermediate state before returning to NORMAL, at a rate governed by the strategy's or scenario's configured recovery-time parameter (Part XXXVIII). The dashboard and 3D twin animate this transition so a viewer can see recovery happening, not just a final state flip.

Part X — What-If Decision Engine

97. Intervention Model

An intervention is modeled as a set of parameter overrides applied to specific nodes for a specified duration: e.g., "reduce Administration load to 20% of normal" or "redirect UPS priority to ICU bus." The What-If Simulator applies these overrides to a copy of the current state, then re-runs the same cascade and risk logic forward across a configured time horizon — it does not use a separately coded simulation path per strategy.

98. Strategy A — Do Nothing

Baseline: no operator action. Used as the comparison floor for every other strategy, and to make the value of intervention visible in the demo.

99. Strategy B — Activate Backup Generator

Models bringing the generator online (if not already active) and connecting it to the emergency bus at its configured ramp-up rate.

100. Strategy C — Generator + Reduce Non-Critical Load

Combines Strategy B with a configured percentage reduction in non-critical (e.g., Administration, non-critical lighting) load, freeing backup capacity for critical services — this is the load-shedding scenario called out in Section 5.6 as particularly important for demonstrating the value of the non-critical-load model.

101. Strategy D — Redistribute Loads

Models shifting load priority between buses/services without necessarily shedding any load outright — e.g., temporarily lowering General Ward's allocated capacity share to protect ICU/OT/Emergency during a capacity shortfall.

102. Strategy E — Prioritize Critical Services

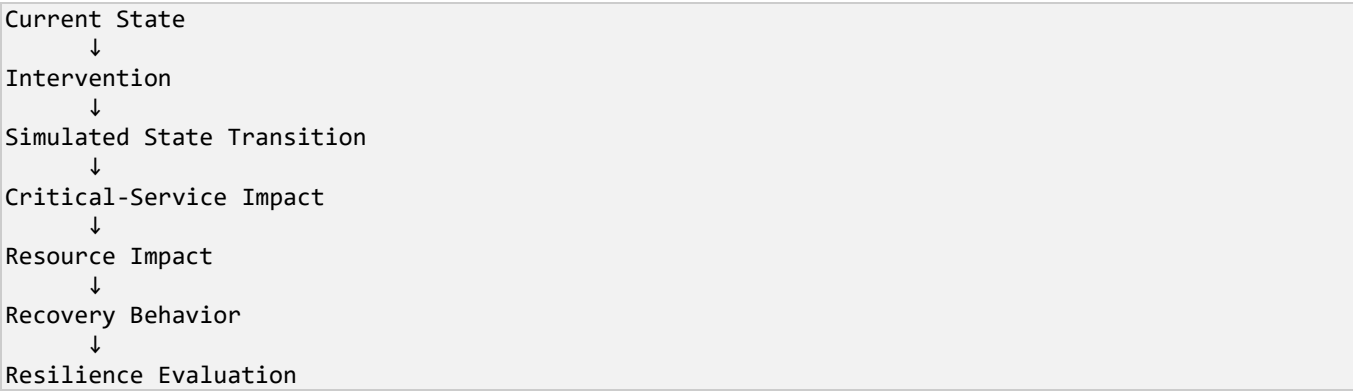
Models an explicit priority override that guarantees ICU/OT/Emergency receive available backup capacity first, with General Ward and non-critical loads receiving only the remainder.

103. Strategy F — Combined Strategy

Applies generator activation, load shedding, redistribution, and critical-service prioritization together, representing the most aggressive coordinated response the simulator supports.

104. Scenario Comparison

For each strategy, the simulator reports the same set of comparable outcome metrics, computed at the same simulated time horizon:



Metric	What it captures
Critical-service continuity	Fraction/degree of ICU, OT, and Emergency service maintained across the simulated horizon.
Backup margin	Remaining backup capacity (generator headroom, battery reserve) at the end of the horizon.
Recovery time	Simulated time until affected nodes return to NORMAL under this strategy.
Resource impact	Load shed, fuel/battery consumption, or other modeled resource cost of the strategy.
Resilience Index	The Part XI composite score, computed at one or more points along the simulated horizon.

105. Optimization / Evaluation

Strategy comparison is not reduced to a single "winner" label without qualification. The system reports "best-performing simulated response under the defined prototype objectives" rather than "the safest response," and the underlying objective function and its weights are fully specified and adjustable in Part XLV, so a judge or operator can see exactly why one strategy outranked another for a given scenario, and can recognize that a different weighting could change the ranking.

Part XI — Resilience

106. Resilience Definition

"Resilience," as used throughout this document, means the modeled system's ability to maintain critical hospital-service continuity through a disruption and return to normal operation, given its configured infrastructure, backup capacity, and available interventions. It is a project-defined engineering usage of the term, not a reference to any certified hospital-resilience or clinical-safety standard.

107. Resilience Index

A single normalized score, 0–100, summarizing modeled infrastructure/service health for a given state, with four qualitative bands:

Band	Range	Meaning
STABLE	80–100	All modeled critical services at or near full continuity, ample backup margin.
DEGRADED	55–79	Some service or backup margin reduction, no critical service at high risk.
AT RISK	30–54	At least one critical service showing meaningfully elevated risk.
CRITICAL	0–29	One or more critical services at high modeled risk of disruption.

Illustrative example only — not an experimental result:

State	Illustrative Resilience Index
Normal	94 / 100
After Failure	43 / 100
Strategy B applied	76 / 100
Strategy C applied	91 / 100

108. Mathematical Formulation

The Resilience Index R is defined as a weighted combination of normalized sub-scores, each in $[0, 1]$, scaled to 0–100:

$$R = 100 \times (w1 \cdot C + w2 \cdot A + w3 \cdot B - w4 \cdot T_{\text{norm}} - w5 \cdot U_{\text{norm}})$$

where:

- C = critical-service continuity (weighted avg. of 1 - risk_i across ICU/OT/Emergency, weighted by service criticality)
- A = infrastructure availability (fraction of modeled assets in NORMAL or DEGRADED, not AT RISK/FAILED)
- B = backup margin remaining (generator headroom + battery reserve, normalized to configured capacity)
- T_{norm} = normalized recovery time (time-to-NORMAL relative to a configured reference recovery time, capped at 1)
- U_{norm} = normalized resource usage (load shed / resources consumed, relative to a configured reference, capped at 1)

$w1..w5$ = configurable non-negative weights, $\text{sum}(w1..w3) - \text{sum}(w4..w5)$
normalized so $R \in [0, 100]$ for all valid input combinations

Baseline weights (prototype assumption, tunable): $w1$ (continuity) = 0.40, $w2$ (availability) = 0.20, $w3$ (backup margin) = 0.15, $w4$ (recovery time) = 0.15, $w5$ (resource usage) = 0.10. These weights reflect a design judgment that critical-service continuity should dominate the score; they are not derived from any external standard and are exposed as configuration so the team can adjust them during development or in response to judge feedback.

109. Qualitative States

The four bands above are used consistently across the dashboard, the timeline, and the what-if comparison view, so a viewer only needs to learn one scale.

110. Service Continuity

C (critical-service continuity) is itself a criticality-weighted average of $(1 - \text{risk})$ across ICU, OT, and Emergency (Part XVIII defines how per-service risk is computed from infrastructure state). General Ward and non-critical loads are intentionally excluded from C's numerator by default, reflecting the modeled priority ordering — but their state still feeds into A (infrastructure availability) and U (resource usage), so degrading them is not "free" in the score even though it is not penalized in C specifically.

111. Recovery

Recovery time (T) is measured from the point an intervention is simulated as applied to the point all previously AT RISK/FAILED nodes return to NORMAL under that intervention's configured recovery-rate parameters (Part XXXVIII).

112. Limitations (Resilience Index)

- The Resilience Index is a project-defined prototype decision-support metric and is not a standardized hospital safety or clinical metric.
- Weights are a design choice, not derived from empirical calibration against real incident data (none exists for this project).
- The score is only as good as the underlying state estimates and thresholds, which are themselves configurable prototype parameters.
- A single scalar score necessarily discards information; the dashboard therefore always shows the sub-scores (C, A, B, T, U) alongside the composite, not the composite alone.

Part XII — User Interface

113. Dashboard

The dashboard is one of the project's primary presentation strengths and is designed around a single hierarchy a judge can absorb in seconds (Section 29):

```

CURRENT STATE
↓
ACTIVE INCIDENT
↓
AFFECTED INFRASTRUCTURE
↓
AFFECTED SERVICES
↓
PREDICTED CASCADE
↓
RESPONSE OPTIONS
↓
RESILIENCE IMPACT

```

Current-state panel: Resilience Index, infrastructure health summary, critical-service status, active alerts. Live-telemetry panel: power, generator, UPS, battery, water, HVAC, medical gas. Dependency panel: what depends on what (a simplified, dashboard-friendly view of the full Dependency View). Incident panel: what failed, what is affected, what may happen next, and why. Simulation panel: strategies A–F. Explanation: a concise "Why?" interaction (Part XIX/XLIV). The dashboard is deliberately not overcrowded — clarity is prioritized over decoration (Section 29).

114. Live Twin

The 3D hospital view plus current state, described fully in Part XXVII.

115. Dependency View

A graph visualization of infrastructure and service nodes and their relationships (Section 9, Part VI), letting the operator answer "what depends on what" directly rather than inferring it from separate panels.

116. Incident View

Shows the active failure, the cascade path that produced current risk levels, and the incident timeline (Section 21/47).

117. Timeline

A time-scrubbing control across the incident's simulated timeline (T+0, T+5, T+10, T+20, ...), showing changing backup margin, service risk, infrastructure state, and Resilience Index at each point (Section 21). These times are illustrative and calculated/configured by the simulation model, never presented as real hospital predictions.

118. What-If Lab

Runs and compares Strategies A–F (Part X) side by side.

119. Explanation

A concise "Why?" interaction returning graph/simulation-grounded causal text (Part XLIV), e.g.:

```
Why is ICU at risk?

Transformer T1 failed
↓
Emergency bus capacity decreased
↓
ICU depends on this bus
↓
Available capacity approaches configured threshold
↓
ICU service risk increased

Why was Strategy C recommended?

• Preserves ICU service
• Preserves Emergency service
• Maintains OT operation
• Reduces non-critical demand
• Maintains larger backup margin
```

120. Recovery View

Shows the digital twin and dashboard metrics animating back toward NORMAL after a strategy is applied (simulated), closing the failure → cascade → intervention → recovery narrative (Part XX).

121. UX Principles

- A judge should understand the current incident and available responses without a verbal walkthrough.
- Beautiful design is valuable, but clarity is more important than decoration — the dashboard is not judged on card count.
- The same underlying state drives the 2D dashboard and the 3D twin, so the two views never visually disagree.
- Only four core views are recommended (Live Twin, Dependency View, Incident View, What-If Lab); additional views are added only if they clearly improve the operator journey, not to appear feature-rich (Section 50).

49. User Journey (cross-referenced from Part III)

Primary user: a hospital infrastructure/operator persona. The user can: (1) view current infrastructure state; (2) inspect critical services; (3) inspect dependencies; (4) trigger a simulated incident; (5) watch the cascade; (6) inspect predicted impact; (7) run what-if scenarios; (8) compare response strategies; (9) inspect explanation; (10) view simulated recovery. No additional user roles are introduced for the prototype.

Part XIII — Software Engineering

122. Technology Stack

Layer	Technology	Why
Frontend	React	Team familiarity, large ecosystem, natural fit with React Three Fiber for the 3D layer.
3D	Three.js / React Three Fiber	Mature WebGL abstraction; React Three Fiber lets the 3D scene consume the same state/props model as the rest of the React app.
Backend	FastAPI (Python)	Async-friendly, fast to develop in, native WebSocket support, and shares a language with the simulation/ML stack.
Real-time communication	WebSockets	Push-based updates avoid polling latency/overhead for a live-feeling dashboard.
Relational database	PostgreSQL	Mature, reliable store for time-series telemetry, scenario/simulation records, and configuration metadata.
Dependency graph	Neo4j	Native traversal for variable-depth dependency queries (Part VI).
ML	Python + scikit-learn	scikit-learn ships Isolation Forest and standard preprocessing utilities out of the box; no need to hand-roll these for a hackathon timeline.
Simulation	Python	Shared language with backend/ML avoids a cross-language integration layer.
Visualization	React charting libraries (e.g., Recharts/Chart.js class tooling)	Standard, well-documented options for the 2D dashboard's charts.
Deployment	Containerized (Docker)	Reproducible local dev and a straightforward path to any container-friendly host.

This stack is used unless a specific implementation constraint discovered during the build makes a component impractical, in which case the substitution and reason should be recorded rather than silently made.

123. Repository Structure

```

resilienceos/
├── frontend/           # React app (dashboard + embedded 3D twin)
├── backend/            # FastAPI app: REST + WebSocket endpoints
├── simulation/         # State machine, cascade engine, what-if engine
├── ml/                 # Anomaly detection, risk estimation
├── graph/              # Neo4j schema, seed scripts, Cypher query modules
├── database/           # PostgreSQL schema/migrations
├── models/             # Shared data models/schemas (Pydantic, TS types)
├── scenarios/          # Configured hospital models & incident scenarios
├── tests/              # Unit, integration, scenario tests
├── docs/               # This document and supporting specs
├── scripts/            # Dev/setup/seed scripts
├── docker/             # Dockerfiles, compose configuration
└── README.md

```

124. API Architecture

The API is proposed, not yet fully implemented; endpoints are marked PROPOSED until confirmed working, per Part XIX.

Endpoint	Method	Purpose
/hospital/state	GET	Current aggregate infrastructure + service state and Resilience Index.
/assets	GET	List configured infrastructure assets and their current state.
/services	GET	List configured hospital services and their current status.
/dependencies	GET	Return the dependency graph (nodes + relationships) for the Dependency View.
/telemetry	GET	Recent telemetry history for a given asset.
/scenarios	POST	Register/configure a new incident scenario.
/failures/inject	POST	Trigger a failure injection (asset, severity, duration).
/simulation/run	POST	Run a what-if simulation for one or more strategies against a given state.
/simulation/{id}	GET	Retrieve results for a previously run simulation.
/resilience/{id}	GET	Retrieve the Resilience Index breakdown for a given state/simulation id.
/explanations/{id}	GET	Retrieve the generated explanation for a given risk value or recommendation id.
/ws/telemetry	WS	Streaming live telemetry updates.
/ws/twin	WS	Streaming twin/state updates for 3D and dashboard synchronization.

Example: POST /failures/inject

Request:

```
{
  "asset_id": "TRANSFORMER-01",
  "severity": "high",
  "duration_minutes": 20
}
```

Response (200):

```
{
  "incident_id": "inc_0007",
  "status": "propagating",
  "affected_assets": ["EMERGENCY-BUS-01"],
  "affected_services": []
}
```

Error (422): asset_id not found in current hospital configuration.

This request/response shape is a design proposal illustrating the contract, not a captured output from a running deployment.

125. Database Design

Full conceptual data model given in Part XXV. Summary of what belongs where: PostgreSQL holds telemetry time series, scenario/simulation run records, resilience-index history, and alerts; Neo4j holds the dependency graph itself (assets, services, relationships). Asset/service identifiers are shared keys between the two stores so a telemetry row or simulation record can be joined back to its graph node without duplicating the graph structure into PostgreSQL.

126. Neo4j Schema

Specified in Part VI/XXXVI: node labels (Infrastructure, Service), relationship types (SUPPLIES, FEEDS, POWERS, BACKS_UP, DEPENDS_ON, COOLS, PROVIDES_WATER, PROVIDES_GAS, SUPPORTS/AFFECTS), and their properties.

127. WebSocket Architecture

Two logical channels — `/ws/telemetry` (raw sensor-style updates) and `/ws/twin` (higher-level state/incident updates for rendering) — are kept separate so a client that only needs twin state does not have to filter a noisier raw telemetry stream, and vice versa. Both channels use a small JSON envelope with an event type, target node id, and payload, so the frontend can dispatch updates generically rather than needing a bespoke handler per event type.

128. Deployment

Local development: docker-compose bringing up FastAPI, PostgreSQL, Neo4j, and the React dev server together. Cloud deployment: containerized services on a practical, reliable hosting provider chosen based on cost and reliability constraints at build time — no specific provider or URL is committed to in this document, and no deployment success is claimed unless and until it has actually occurred.

129. Configuration

Hospital topology, thresholds, criticality, and scenario definitions live in versioned configuration files (Part XXIV), not hard-coded constants, so the same codebase can represent a different hospital model.

130. Logging / Observability

Key events — failure injection, each cascade propagation step, each what-if simulation run, and each Resilience Index computation — are logged with enough structure (timestamp, affected node ids, triggering event) to reconstruct exactly what happened during any given demo run after the fact.

Part XIV — Testing & Evaluation

131. Test Strategy

Testing is organized into four layers — unit, integration, scenario, and edge-case — described below. No pass/fail results are asserted in this document; this section specifies what will be tested and what correct behavior looks like, not results that have not yet been produced.

132. Unit Tests

- Graph propagation: given a small fixture graph, does traversal reach exactly the expected downstream nodes?
- Threshold logic: does a node transition state exactly when its computed margin crosses its configured threshold, in both directions?
- State transitions: does the four-state machine (NORMAL/DEGRADED/AT RISK/FAILED) reject invalid transitions?
- Resilience calculation: does the formula in Part XLV return the expected value for hand-computed fixture inputs?
- Simulation: does a no-op strategy ("Do Nothing") reproduce the same trajectory as the unmodified cascade engine?

133. Integration Tests

- FastAPI ↔ Neo4j: a traversal query issued through the API returns the same nodes as issuing it directly against the graph.
- FastAPI ↔ PostgreSQL: a telemetry write is retrievable via the /telemetry endpoint with matching values.
- Simulator ↔ backend: a simulator tick results in the expected state-engine update.
- Backend ↔ WebSocket: a state change is broadcast to a connected test client within the target latency.
- Backend ↔ frontend: a failure injected via the UI results in the dashboard showing the expected cascade.
- Backend ↔ 3D twin: a state change is reflected in the 3D scene's rendered material/state within one visual update cycle.

134. Scenario Tests

- Main power failure (hero scenario, Part XX).
- Generator failure (Section 90).
- UPS/battery degradation (Section 91).
- Water pump failure (Section 92).
- HVAC failure (Section 93).
- Medical-gas pressure drop (Section 94).

135. Edge Cases

- Multiple simultaneous failures (Section 95).
- Insufficient backup capacity to cover configured critical load.

- Missing telemetry (a node stops reporting) — the state engine should mark it stale/unknown rather than silently freezing its last value indefinitely.
- Delayed telemetry — out-of-order timestamps should not corrupt the state history.
- A disconnected graph component (a node with no path to any service) — cascade traversal should simply not reach it, not error.
- Recovery after intervention, including repeated intervention (applying a second strategy mid-recovery from a first).
- An invalid scenario configuration (e.g., a service with no declared dependencies) — the system should flag this at load time rather than fail silently during a demo.

136. Performance Evaluation

Proposed evaluation methodology (no results reported yet): measure cascade-detection time (origin failure to fully propagated state) and dashboard update latency (state change to rendered UI update) across the scenario-test set once implemented, and record these as engineering benchmarks distinct from any resilience or risk metric.

137. ML Evaluation

See Part VIII, Section 84 — to be evaluated after implementation, on synthetic held-out scenarios only.

138. Simulation Evaluation

Proposed methodology: for each of the six strategies, confirm the simulator's reported outcome direction matches the intended mechanism of that strategy (e.g., Strategy C should show lower non-critical load and, in most configured scenarios, higher backup margin than Strategy A) — a sanity/consistency check, not a claim of optimality.

139. Usability Evaluation

Qualitative evaluation criteria: judge comprehension (can a first-time viewer explain the current incident after seeing the dashboard for under a minute), operator interpretability of the explanation text, and visualization clarity of the 3D cascade. No formal user study has been conducted; this is a proposed evaluation lens for internal team review and demo rehearsal, not a reported result.

Part XV — Security & Safety

140. Prototype Security Boundary

The prototype operates entirely on synthetic data in a controlled environment: no patient data of any kind is used or stored, no connection to real hospital infrastructure or actuators exists, and all telemetry is generated by the simulator described in Part VII. Given this, a lightweight security posture appropriate to a demo environment is used rather than a production-grade one, and this document does not overstate what is implemented (Section 59).

141. Patient-Data Boundary

ResilienceOS is, by design, infrastructure-focused and patient-data-free (Section 19). No patient identifiers, clinical records, or care data are modeled, stored, or displayed anywhere in the system. Risk indicators are expressed as Critical Service Risk, Service Disruption Risk, and Critical-Service Continuity — never as a percentage of patients affected or any other patient-level statistic.

142. Human-in-the-Loop

ResilienceOS is a decision-support system, not an autonomous infrastructure-control system. It analyzes, predicts, simulates, compares, and explains; it does not autonomously actuate any real or simulated hospital equipment. Every consequential action — injecting a failure, running a simulation, applying a strategy — requires explicit operator input (FR-06, FR-12, FR-18). The operator remains responsible for any real-world intervention; the system's output is an input to that decision, not a substitute for it.

143. Future Production Security

If a future version of this project were to move toward real BMS/IoT integration (Part XVI), the following would need to be designed and implemented — none of it exists in the prototype, and none of it is claimed here as implemented:

- Authentication and role-based access control (RBAC) for operator accounts.
- Secure telemetry ingestion (encrypted transport, authenticated sources).
- Encryption at rest for any stored telemetry or configuration.
- Audit logging of all operator actions and system-generated recommendations.
- Network segmentation between the decision-support system and any real control-plane systems.
- Formal cybersecurity validation and, likely, a secure integration pattern with the target facility's BMS rather than direct actuation access.
- Access control policies aligned with the facility's own governance, not just the application's own login system.

Part XVI — Feasibility & Impact

144. Technical Feasibility

Every component in the architecture (Part IV) uses mature, well-documented technology (React, FastAPI, PostgreSQL, Neo4j, scikit-learn, Three.js) with substantial existing tooling and community support. The main technical risk is integration effort across five moving parts (frontend, backend, graph, simulation, 3D) within a compressed timeframe, not the feasibility of any individual component in isolation (Part XIX, Technical Risks).

145. Hackathon Feasibility

The Core MVP (Part XX, Section 91) is scoped specifically to be achievable by a six-person student team within a hackathon build window, by deliberately deferring anything not required for the end-to-end failure-to-recovery loop (Part XVII roadmap, Part XIX Section 90, Do Not Overengineer).

146. Scalability

The architecture is designed to scale conceptually from a prototype covering one hospital wing to a larger configuration — more assets, more services, more buildings — without a structural redesign, because the hospital model is configuration-driven (Part XXIV) and the graph/traversal approach does not assume a fixed topology. Actual performance at large scale (hundreds or thousands of nodes) has not been tested and is explicitly a Future Extension concern.

```

Prototype
↓
One hospital wing
↓
Multiple hospital buildings
↓
Multiple utility plants
↓
Multiple facilities
↓
Regional healthcare infrastructure

```

Potential future integration points (Future Extension, not implemented): BMS, SCADA, IoT gateways, MQTT, OPC UA, facility-management systems, and other digital-twin platforms.

147. Sustainability

As a hackathon prototype, ongoing maintenance is not currently funded or staffed beyond the six-person team's own interest; the repository structure (Part XXIII) and configuration-driven design are chosen partly so that a future contributor — inside or outside the original team — could pick the project up without needing to understand undocumented hard-coded assumptions.

148. Real-World Integration

No real-world integration exists today. A realistic future path would involve: (1) partnering with a facility willing to share anonymized, non-patient infrastructure telemetry; (2) replacing the synthetic simulator with a real

telemetry adapter while keeping the graph/cascade/simulation logic unchanged; (3) validating configured thresholds and dependency topology against that facility's actual as-built systems and applicable codes (e.g., NFPA 99/110 for U.S. facilities); and (4) a formal security and safety review before any read-write connection to real control systems is considered — full actuation access is not a design goal even in this future path, given the human-in-the-loop principle (Part XV).

149. Potential Impact

Within the scope of what has actually been built and tested, the honest impact claim is a working proof of concept demonstrating that dependency-aware cascade reasoning and intervention simulation can be built and visualized for a representative hospital model. No measured real-world impact (downtime avoided, cost saved, risk reduced) is claimed, because none has been measured.

150. Deployment Roadmap

See Part LXI (Future Roadmap) for the staged progression from prototype to potential real-world deployment, and Part XVII for the hackathon-internal development roadmap.

Part XVII — Implementation

151. Development Roadmap

Sixteen phases, each with objective, key tasks, dependencies, deliverables, and completion criteria. Phases are ordered so that the end-to-end demo loop (Part XX) becomes possible as early as feasible, with polish and secondary features layered on afterward — consistent with the Demo-First Engineering principle (Section 80).

Phase	Objective	Key deliverable	Completion criteria
0 — Architecture	Agree on module boundaries and data contracts.	Architecture doc (this document, Part IV) + interface stubs.	All six team members can state which module owns which responsibility.
1 — Data model	Define configuration schema for assets/services/dependencies.	JSON/YAML config schema (Part XXIV).	A sample hospital config validates against the schema.
2 — Hospital model	Populate the baseline hospital (Part V).	Seed configuration for ICU, OT, ED, Ward, Plant.	Config loads without error and produces the expected node/edge counts.
3 — Graph	Stand up Neo4j and load the config as a graph.	Working graph + seed script.	A sample Cypher traversal returns expected downstream nodes.
4 — Simulator	Build the stateful telemetry simulator (Part VII).	Simulator producing state-driven telemetry.	Telemetry values visibly respond to a manually triggered state change.
5 — Cascade engine	Implement failure propagation (Part XVII pseudocode below).	Cascade engine module.	Injecting a failure produces the expected downstream state changes on the seed scenario.
6 — Resilience Index	Implement the Part XI formula.	Resilience-index module.	Formula output matches hand-computed fixture values.
7 — What-if engine	Implement Strategies A–F (Part X).	What-if simulator module.	All six strategies run without error and return comparable metrics.
8 — Backend APIs	Expose REST endpoints (Part XXIV/XXXVII).	FastAPI app with core endpoints.	Endpoints return expected shapes against the seed scenario.
9 — Real-time system	Wire up WebSocket streaming.	/ws/telemetry and /ws/twin channels.	A connected client receives an update within target latency of a state change.
10 — 3D twin	Build the medium-detail 3D hospital (Part XXVII).	React Three Fiber scene.	Scene renders and updates from live state, not static data.
11 — Dashboard	Build the 2D dashboard views (Part XII).	Dashboard UI.	All four core views are navigable and populated from live state.
12 — ML	Implement anomaly detection and risk estimation (Part VIII).	ML module.	Anomaly flag fires on induced-failure runs in the synthetic test set.

Phase	Objective	Key deliverable	Completion criteria
13 — Explainability	Implement the templated explanation layer (Part XLIV).	Explanation module.	"Why?" queries return text traceable to the actual graph path/simulation output.
14 — Integration	Connect all modules end-to-end.	Working failure-to-recovery loop.	The Core MVP loop (Part XX) runs without manual intervention between steps.
15 — Testing	Execute the test strategy (Part XIV).	Test suite + edge-case pass.	Scenario tests and edge cases behave as specified, without crashes.
16 — Demo prep	Rehearse the hero demo (Part XVIII) and prepare fallback.	Demo script + fallback recording/dataset.	Full demo runs twice consecutively without manual fixes.

152. Team Responsibilities (Proposed, not final)

Proposed role	Primary responsibility
System architecture / backend	FastAPI app, API design, integration across modules.
Digital twin / 3D	React Three Fiber scene, twin-state synchronization.
Graph / data	Neo4j schema, seed data, PostgreSQL schema, configuration model.
ML / simulation	Cascade engine, what-if simulator, anomaly/risk models.
Frontend / UX	Dashboard views, information architecture, styling.
Integration / testing / documentation	End-to-end wiring, test suite, this document's upkeep, demo rehearsal.

These are proposed responsibilities to help the team start, not final assignments — actual allocation should reflect each member's strengths and availability.

153. Milestones

- M1: Graph + simulator running independently and producing sensible state changes (end of Phase 4–5).
- M2: Cascade engine correctly propagates the hero power-failure scenario end to end (end of Phase 5).
- M3: All six what-if strategies runnable and comparable (end of Phase 7).
- M4: Full-stack demo loop (inject → cascade → simulate → compare → explain → recover) runs without manual intervention (end of Phase 14).
- M5: Demo rehearsed twice consecutively without a manual fix, with a working fallback prepared (end of Phase 16).

154. Definition of Done

A phase is "done" when its stated completion criterion (table above) is met and the corresponding module has at least the unit/integration tests specified for it in Part XIV. A feature is not "done" merely because it renders visually — it must be driven by the actual simulation/graph state, not a hard-coded placeholder, before Phase 14 (Integration) is considered complete for that feature.

155. Demo Preparation

See Part XVIII for the full demo script, timing variants, and fallback plan.

Pseudocode — Core Algorithms

Failure injection

```
function inject_failure(asset_id, severity, duration):
    asset = graph.get_node(asset_id)
    asset.state = compute_state_from_severity(severity)
    asset.failure_end_time = now() + duration
    event_queue.push(FailureEvent(asset_id, now()))
    return incident_id
```

Dependency traversal

```
function traverse_downstream(origin_ids, max_depth=5):
    visited = {}
    frontier = origin_ids
    for depth in range(max_depth):
        next_frontier = []
        for node in frontier:
            for edge in graph.outbound_edges(node):
                if edge.target not in visited:
                    visited[edge.target] = (node, edge, depth+1)
                    next_frontier.append(edge.target)
        frontier = next_frontier
    return visited # target -> (source, edge, depth) for explanation path
```

Cascade propagation

```
function propagate_cascade(origin_ids):
    reachable = traverse_downstream(origin_ids)
    changed = []
    for target, (source, edge, depth) in reachable.items():
        available = compute_available_capacity(target, edge)
        if available < target.threshold:
            new_state = derive_state(available, target.threshold)
            if new_state != target.state:
                target.state = new_state
                changed.append((target, source, edge))
    for service in graph.servicesDepending_on(changed):
        update_service_impact(service) # Part XVIII logic
    return changed
```

State update

```
function derive_state(available_fraction, threshold):
    if available_fraction >= 1.0: return NORMAL
    if available_fraction >= threshold: return DEGRADED
    if available_fraction >= threshold * 0.5: return AT_RISK
    return FAILED
```

Risk estimation

```
function estimate_service_risk(service):
    upstream_margin = min(dep.available_fraction for dep in service.dependencies)
    weighted_degradation = sum(
```

```

        dep.strength * (1 - dep.available_fraction) for dep in service.dependencies
    )
    risk = clamp(weighted_degradation - service.redundancy_offset, 0, 1)
    return risk

```

Time-to-threshold

```

function time_to_threshold(current_reserve, depletion_rate, threshold):
    if depletion_rate <= 0: return None    # not depleting
    remaining = current_reserve - threshold
    return max(remaining / depletion_rate, 0)

```

What-if simulation

```

function run_strategy(state_snapshot, strategy, horizon_minutes):
    sim_state = deep_copy(state_snapshot)
    apply_strategy_overrides(sim_state, strategy)
    trajectory = []
    for t in range(0, horizon_minutes, STEP):
        advance_state_machine(sim_state, STEP)
        propagate_cascade(sim_state.active_failures)
        trajectory.append(snapshot_metrics(sim_state, t))
    return summarize(trajectory)    # continuity, margin, recovery time, resource cost

```

Resilience Index

```

function resilience_index(state):
    C = weighted_continuity(state.critical_services)
    A = fraction_available(state.assets)
    B = normalized_backup_margin(state)
    T = normalized_recovery_time(state)
    U = normalized_resource_usage(state)
    R = 100 * (W1*C + W2*A + W3*B - W4*T - W5*U)
    return clamp(R, 0, 100)

```

Response comparison

```

function compare_strategies(state, strategies):
    results = {s: run_strategy(state, s, HORIZON) for s in strategies}
    scored = {s: objective_score(r) for s, r in results.items()}
    ranked = sorted(scored.items(), key=lambda kv: kv[1], reverse=True)
    return results, ranked    # UI shows all results; ranking is a label, not an override

```

Explanation generation

```

function explain_risk(service):
    path = shortest_causal_path(service)    # from cascade result, not re-derived
    lines = [render_step(step) for step in path]
    return join(lines, ' \u2193 ')    # rendered as an arrow chain in the UI

```

Part XVIII — Demo

156. Demo Story

The demo is built as a story, not a feature tour: Failure → Cascade → Intervention → Recovery. Every step exists to advance that narrative; steps that do not are cut.

157. Demo Timeline / 158. Main Power Failure Walkthrough

10. Open dashboard. Brief framing: hospitals contain interconnected infrastructure systems; monitoring can show that something failed, but the important question is what that failure could affect next.
11. Show normal hospital: live telemetry, infrastructure status, critical services, Resilience Index, 3D twin.
12. Inject failure: click "SIMULATE MAIN POWER FAILURE." Show Grid → FAILED immediately.
13. Show immediate response: Generator → STARTING, UPS → ACTIVE, Critical Load → CHANGING.
14. Pause and progress through time: T+5, T+10, T+20 minutes, with changing backup margin, service risk, infrastructure state, and Resilience Index.
15. Show dependency cascade visually: Power → Transformer → Emergency Bus → Generator/UPS → HVAC/other dependencies → ICU/OT/Emergency.
16. Ask: "What can we do?" — open the What-If Lab.

159. What-If Walkthrough

17. Run the six intervention scenarios (Part X).
18. Compare: critical-service continuity, backup margin, recovery time, resource impact, resilience index across strategies.
19. Explain: click "Why?" to show concise causal reasoning for the top-performing strategy.
20. Apply the selected simulated strategy and show the digital twin recovering.

Final message: the demo should visually communicate Failure → Cascade → Intervention → Recovery, and feel like a story, not a software feature tour.

160. Judge Experience

Designed so a judge understands the core loop within the first two minutes of the demo, before any technical stack discussion begins. Technical depth is available on request (Q&A, Part XIX) but is not front-loaded into the opening narrative.

161. Backup Demo Plan / 162. Failure Recovery Plan

If live services fail during the hackathon (network issues, a broken build, venue Wi-Fi problems), a fallback mode uses deterministic prerecorded scenario data driving the exact same UI and event sequence, so the audience sees the identical failure → cascade → intervention → recovery story even if the live backend is unavailable. The fallback is prepared in Phase 16 (Part XVII) and rehearsed alongside the live version, not improvised on the day.

82. Hackathon Presentation Strategy

Format	Content focus
30-second explanation	The positioning statement (Section 5): "Normal monitoring tells you something failed; ResilienceOS tells you what that failure could affect next and lets you test what to do about it."
1-minute explanation	Add the central question (Section 5) and name the four pillars: dependency graph, cascade prediction, what-if simulation, human decision.
3-minute demo	Steps 1–4 and 7–10 of the hero demo (skip most of the timeline scrubbing).
5-minute demo	Full hero demo, Steps 1–11.
10-minute technical presentation	Full demo plus architecture (Part IV), Resilience Index formulation (Part XI), and 2–3 anticipated judge questions (Part XIX) answered proactively.

Every format preserves the same central narrative; the pitch is not restructured into a list of technologies at any length.

83. Judge Memory Test

What a judge should remember thirty minutes later: "They built a virtual hospital infrastructure where I can trigger a failure, watch the cascade happen, test different responses, and see which response preserves critical services." Every design decision in this document is checked against whether it helps or dilutes this one sentence.

Part XIX — Critical Review

163. Known Limitations

- Synthetic telemetry only — no real hospital infrastructure data has been used or accessed.
- No real-world validation of the model, thresholds, or Resilience Index against an actual incident.
- Simplified infrastructure — the modeled hospital is deliberately small (Part V) and does not represent every subsystem a real facility would have.
- Configurable assumptions throughout (capacities, thresholds, delays, criticality scores) are prototype parameters, not measured or certified values.
- Limited hospital scale — one small wing-equivalent model, not a full facility.
- Limited failure scenarios — six named strategies and a handful of failure types; real incidents can be more varied and combinatorial.
- The Resilience Index is a prototype metric with no external validation or standardization.
- No certified operational thresholds are used or claimed.
- No real-world BMS/SCADA integration exists.
- Simplified backup architecture relative to real hospital essential-electrical-system designs.
- Probabilistic risk/anomaly outputs carry real uncertainty and have not been benchmarked against real-world ground truth.
- No autonomous infrastructure control is implemented or intended.
- No clinical decision-making capability exists or is claimed.

164–167. Risks

Risk category	Specific risk	Mitigation
Technical	Integration across five stacks (frontend, backend, graph, simulation, 3D) under time pressure.	Demo-First Engineering (Section 80): prioritize the minimum end-to-end loop before polish; integrate early and often (Phase 14 sits mid-roadmap intentionally, not at the very end).
Technical	WebSocket real-time updates may be flaky on hackathon venue networks.	Prepare the deterministic fallback mode (Section 161) well before the event.
Modeling	Threshold/capacity values, chosen without domain-expert review, may look implausible to an engineering-literate judge.	Be transparent that these are configurable prototype parameters (this document's labeling convention) and be ready to justify the mechanism, not the specific number.
Modeling	Six-strategy comparison may look arbitrary if the objective function's weights are not explainable on demand.	Rehearse the Section 108 formula and weight rationale as part of Q&A prep.
Data	No real dataset exists for ML validation, which a data-science-literate judge will probe.	Answer honestly: synthetic-only, with a proposed evaluation methodology, not fabricated metrics (Part VIII, Section 84).
Team/scope	Overbuilding — attempting every capability in this document before the demo loop works.	Follow the MVP/Essential/Valuable/Future classification (Section 90–91) strictly.

168. Future Improvements

See Part LXI (Future Roadmap) for staged post-hackathon extensions: real BMS/IoT integration, expanded infrastructure coverage, formal ML evaluation on real data, and eventual security/certification work.

169–171. Judge Q&A — Difficult Questions and Defensible Answers

Judge question	Honest answer
Why does this need to exist?	Because hospital infrastructure systems are interdependent, but most monitoring tools report per-system state, not cross-system consequences. ResilienceOS closes that specific gap for a demonstrable prototype scale.
Isn't this just a dashboard?	The dashboard is the interface; the substance is the dependency graph, cascade engine, and what-if simulator that decide what the dashboard shows. A dashboard alone cannot answer "what happens if I do X" — this system can.
Isn't this just a digital twin?	The digital twin (Part XI) is the stateful representation layer; the differentiator is the cascade reasoning and intervention simulation built on top of it (Section 55, Existing System Comparison).
Isn't this just a knowledge graph?	The graph is one component. Cascade logic, simulation, evaluation, and explanation are separate modules that consume the graph; the graph alone does not simulate time-based failure evolution or compare interventions.
Where is the AI?	Anomaly detection (Isolation Forest) and risk estimation are the ML components; deterministic graph/threshold logic and simulation do most of the causal reasoning by design (Part XIV, Hybrid Intelligence) — AI is not asked to do the whole job, on purpose.
What exactly is being predicted?	Time-to-threshold for specific resources (e.g., battery reserve) and a modeled service-risk trajectory under a given scenario — not a clinical or patient outcome.
How is cascade propagation calculated?	Breadth-first traversal of the dependency graph from the failed node, applying capacity/threshold logic at each hop (Part XVII pseudocode); the traversal path is retained and reused directly for explanation.
Where did the data come from?	All telemetry is generated by a configurable, stateful synthetic simulator (Part VII); no real hospital data was used.
Are the sensor values real?	No — they are synthetic and driven by the modeled state machine, not by any physical sensor.
How do you know the recommendation is correct?	"Correct" is scoped to the defined objective function (Part XLV) applied to the simulated outcome, not to a validated real-world ground truth. The system reports the best-performing simulated response under stated objectives, and shows its work.
What does "resilience" mean here?	Modeled ability to maintain critical-service continuity through disruption and return to normal, per the project-defined Resilience Index (Part XI) — not a certified clinical or safety standard.
How is the Resilience Index calculated?	A weighted combination of continuity, availability, backup margin, recovery time, and resource usage (Section 108's formula), with configurable weights.

Judge question	Honest answer
Why Neo4j? Why not a normal database?	Because the central operation is variable-depth dependency traversal, which graph databases handle natively and relational recursive queries handle more awkwardly (Part VI).
Why do you need 3D?	To make the cascade visible rather than only readable — a judge should be able to see the chain of consequence, not just read a list of affected node ids.
What happens if two systems fail simultaneously?	The cascade engine seeds traversal from multiple origin nodes; per-node threshold logic is unaffected by how many origins triggered it (Section 95).
What happens if backup systems fail?	Modeled as a second failure on the generator/UPS/battery nodes themselves, propagated by the same engine — no special-cased logic is required (Section 90–91).
Can it work with real hospitals?	Not without further work: real telemetry integration, threshold validation against as-built systems and applicable codes, and a security review (Part XVI, Section 148). The prototype is a proof of concept, not a deployable product.
Is it safe?	It does not control any real infrastructure, so it cannot directly cause an unsafe physical action; its main safety obligation is to avoid misleading an operator, which is why every number is labeled and every recommendation is explainable.
Is it clinically validated?	No, and it does not make clinical claims — it is infrastructure-focused and patient-data-free by design.
Can an AI accidentally recommend a dangerous intervention?	The system's ML components influence a risk score, not the strategy list itself; the six strategies are fixed, human-designed interventions, and applying any of them always requires explicit operator confirmation.
What happens without the LLM?	Nothing essential is lost — the core loop (graph, cascade, simulation, resilience index) runs entirely on deterministic and lightweight-ML logic; a language model, if used at all, only phrases the already-computed explanation in natural language (Part XIV).
What is actually implemented vs. future scope?	See the labeling convention (front matter) applied consistently throughout; Part XX (Core MVP) states the minimum complete demonstrable set explicitly.
What makes this different from existing BMS/SCADA?	BMS/SCADA excel at asset-level monitoring and alarms; they typically do not provide dependency-aware cascade prediction, comparative what-if simulation, or a unified service-level resilience score (Part IX, Existing System Comparison — no claim is made that no BMS anywhere has any of these features).
What makes this different from existing research?	Existing research covers infrastructure interdependency theory, healthcare digital twins, and anomaly detection largely separately; this project's contribution is integrating them into one interactive, hospital-scale, operator-facing prototype (Section 18).
Can a six-person student team actually build this?	Only if scope is disciplined: the Core MVP (Section 91) is deliberately sized for exactly this team, with everything else marked Valuable-if-time-permits or Future Extension.
What is your killer demo?	The main-power-failure walkthrough (Part XVIII): inject a failure, watch the cascade in 3D and on the dashboard, run and compare six responses, get an explanation, apply one, watch recovery.

Anti-Buzzword Audit

Technology	Why it is included (or removed)
Neo4j / knowledge graph	Kept — the central operation (variable-depth dependency traversal) is graph-native; see Section 70 rationale.
Digital twin	Kept, but scoped precisely (Part XI): a prototype twin of a simulated hospital, not a marketing-style claim of twinning a real building.
ML (Isolation Forest, risk estimation)	Kept in a narrow, auxiliary role (Part XIV); not positioned as the system's core reasoning engine.
LLM / natural-language explanation	Kept only as a phrasing layer over already-computed, graph/simulation-grounded content (Part XLIV); not used to invent causal claims.
AI agents / multi-agent systems	Not included. No autonomous agent architecture is used or needed for this scope; adding one would not change any capability described here.
SHAP / LIME	Not included in the MVP (Section 85) — the risk formula is already transparent, so post-hoc attribution would add complexity without adding explanatory value.
Blockchain	Not included. No requirement in this project (data integrity, audit, or otherwise) calls for a distributed ledger; a normal audit log in PostgreSQL suffices.
RAG (retrieval-augmented generation)	Not included. There is no external document corpus the system needs to retrieve from; explanations are grounded in the graph/simulation output directly, not in retrieved text.
Optimization (formal solver)	Not included as a general-purpose optimizer; a bounded set of six named strategies is scored against a defined objective function (Part X/XLV), which does not require a generic optimization framework.

Part XX — Conclusion

172. Conclusion

ResilienceOS demonstrates, at hackathon scale, that a small hospital's electrical, backup-power, water, HVAC, and medical-gas systems can be represented as one dependency-aware model; that a failure in that model can be propagated and explained rather than merely alarmed on; that candidate responses can be simulated and compared before any real action is taken; and that all of this can be made visible to a human operator through a synchronized dashboard and 3D twin, without ever asking that operator to trust an opaque recommendation or hand over control to an autonomous system.

The project does not claim to have solved hospital infrastructure resilience, invented digital twins, or built a clinically validated tool. It claims to have built a focused, honestly scoped, technically defensible prototype of one useful idea: normal monitoring tells you something failed; ResilienceOS helps you understand what that failure could affect next and explore what to do about it before acting.

173. Future Vision

Beyond the hackathon, the logical next steps are, in order: formal internal evaluation of the ML and simulation components against a wider synthetic scenario set; a partnership-based pilot using anonymized real (non-patient) infrastructure telemetry from a willing facility; validation of the dependency model and thresholds against that facility's as-built systems and applicable codes; and, only after those steps, cautious exploration of a read-only integration with a real BMS for state observation — with actuation remaining a human decision indefinitely, consistent with the human-in-the-loop principle that defines this project (Part XV).

Traceability

85. Research-to-Implementation Traceability

Research finding	Design decision	Implementation module	Demo feature
Infrastructure interdependency produces cascading failure (Rinaldi et al., 2001)[1]	Model infrastructure and services as an explicit dependency graph	Neo4j graph (Part VI)	Visual dependency cascade (Part XII)
Hospital electrical systems are organized into criticality-tiered branches/buses (NFPA 99/110)[2]	Model buses (Emergency/Main/Non-critical) as distinct nodes with backup priority	Graph schema + State Engine (Part IV, VI)	Load-shedding what-if strategy C (Part X)
A digital twin requires real, continuous synchronization to be more than a static model (NIST NISTIR 8356)[4]	Build a stateful simulator that continuously drives twin state rather than a static 3D scene	IoT Simulator + State Engine (Part VII)	Live-updating 3D twin (Part XXVII)
Isolation Forest isolates anomalies efficiently without profiling "normal" data (Liu, Ting & Zhou, 2008)[8]	Use Isolation Forest as the primary anomaly-detection method	ML module (Part VIII)	Anomaly flag on dashboard (Part XII)
Healthcare digital-twin research is concentrated on clinical/maintenance use cases, not infrastructure-resilience cascade reasoning[6][7]	Position the project specifically at the cascade-reasoning + intervention-simulation niche	Whole-system architecture (Part IV)	The hero demo narrative (Part XVIII)

86. Requirement-to-Feature Traceability

Requirement	System module	Implementation	Demo evidence	Test
FR-06 Failure injection	Cascade Engine	inject_failure() (Part XVII pseudocode)	"SIMULATE MAIN POWER FAILURE" button (Part XVIII)	Scenario test: main power failure (Part XIV)
FR-07 Cascade propagation	Cascade Engine	propagate_cascade() (Part XVII)	Dependency cascade visualization (Part XII)	Unit test: graph propagation (Part XIV)
FR-08 Service impact analysis	State Engine + Cascade Engine	estimate_service_risk() (Part XVII)	ICU/OT/Emergency risk indicators (Part XVIII)	Unit test: threshold logic
FR-12 What-if simulation	What-If Simulator	run_strategy() (Part XVII)	What-If Lab, Strategies A–F (Part X, XII)	Simulation evaluation (Part XIV)

Requirement	System module	Implementation	Demo evidence	Test
FR-14 Resilience Index	Response Evaluator	resilience_index() (Part XVII)	Resilience Index displayed on dashboard (Part XII)	Unit test: resilience calculation
FR-15 Explainable response	Explainability layer	explain_risk() (Part XVII)	"Why?" interaction (Part XII, XVIII)	Manual/usability evaluation (Part XIV)
FR-16 3D twin synchronization	3D Layer	React Three Fiber scene bound to twin state (Part XXVII)	Live 3D twin in demo (Part XVIII)	Integration test: backend ↔ 3D twin

Claim Audit

Every substantive claim in this document falls into one of five categories. This audit exists specifically to identify anything that could be misread as a validated real-world claim.

Verified external fact (supported by a cited source)

- NIST's NISTIR 8356 defines a digital twin as a virtual representation requiring real, continuous synchronization to be more than a marketing-style claim.[4][5]
- NFPA 99 defines the Essential Electrical System and its criticality categories; NFPA 110 defines emergency-power performance requirements including a widely documented 10-second start requirement for the systems it governs.[2]
- Rinaldi, Peerenboom, and Kelly (2001) established the classification of critical-infrastructure interdependencies (physical, cyber, geographic, logical) and the resulting potential for cascading failure.[1]
- Liu, Ting, and Zhou (2008) introduced Isolation Forest as a linear-time, low-memory anomaly-isolation method.[8]
- Healthcare digital-twin research (Balasubramanyam et al. 2024; Xames & Topcu 2024; Mukherjee et al. 2023) is concentrated on clinical, personalized-medicine, and facility-maintenance applications.[6][7]

Project design decision (defined by the team)

- The choice of Neo4j, PostgreSQL, FastAPI, React, and Three.js as the technology stack (Part XIII).
- The four-state infrastructure state machine (NORMAL/DEGRADED/AT RISK/FAILED) and its transition guards (Part VII, XXXVIII).
- The Resilience Index formula and its default weights (Part XI, Section 108).
- The six named what-if strategies (Part X).
- The decision to keep LLM use, if any, confined to phrasing an already-computed explanation (Part XIV).

Prototype assumption (configured for simulation)

- ICU bed count (5–6), General Ward room count (~10), and all capacity/threshold/delay values.
- Criticality scores (ICU/OT/Emergency = 5, General Ward = 3, Administration = 1).
- Baseline Resilience Index weights (w1–w5, Section 108).
- The specific dependency topology used in the seed hospital configuration (Part V/VI) — presented as one illustrative configuration, not a universal template.

Expected behavior (not yet experimentally validated)

- That the cascade engine correctly reproduces the intended hero scenario end to end (to be confirmed via Part XIV scenario testing).
- That the anomaly detector distinguishes induced-failure runs from normal-operation runs in the synthetic test set (to be evaluated per Part VIII, Section 84).
- That the six strategies rank in the directionally expected order for the baseline scenario (to be confirmed per Part XIV, Section 138).

Future capability (not implemented)

- Real BMS/SCADA/IoT-gateway integration (Part XVI, Section 148).
- Production-grade authentication, RBAC, encryption, and audit logging (Part XV, Section 143).
- Formal ML evaluation against real-world labeled data.
- Multi-facility/regional-scale deployment (Part XVI, Section 146).

Technical Consistency Audit

The following invariants were checked for consistency across this document and should be re-checked whenever the document is edited:

- ICU size is consistently stated as 5–6 beds (Sections 5.1, 55).
- General Ward is consistently ~10 rooms (Sections 5.4, 58).
- The patient-data-free boundary is stated consistently (Sections 19, 59, 141) and no section introduces patient-level data.
- Simulated (not real) IoT telemetry is described consistently (Sections 12, 72, 73) with no section implying real sensor data.
- Neo4j is consistently the graph layer and PostgreSQL the relational layer (Sections 30, 63, 122–126) with no duplicated source of truth.
- WebSockets are consistently the real-time transport (Sections 13, 39, 75, 127).
- The React / Three.js / FastAPI / Python stack is used consistently (Part XIII) with no contradicting technology introduced elsewhere.
- Main power failure remains the hero scenario throughout (Sections 20, 89, 157–158).
- The same six response strategies (A–F) are used consistently in Parts X, XII, XVIII, and the pseudocode.
- The human operator remains in the loop in every section that discusses action-taking (Sections 26, 142, 159).
- The Resilience Index is consistently labeled project-defined, never a certified standard (Sections 24, 106, 112).
- Patient risk and clinical outcomes are never claimed anywhere in the document (Section 19, 141, Ethical Boundary).
- Real-world infrastructure control is never claimed as implemented (Sections 26, 142).
- Deterministic graph/rules and probabilistic ML retain distinct responsibilities wherever discussed (Part XIV, VIII).
- Implemented/Proposed/Future labels are applied consistently per the front-matter convention.
- No fabricated metrics or experimental results appear anywhere in the document (Part VIII Section 84, Part XIV).

Appendix — Diagram Specification Catalog

Diagrams are not embedded as images in this document; instead, each is specified precisely enough for any team member to produce it in Mermaid, PlantUML, or draw.io. Where a diagram's content already appears in this document as an ASCII/text schematic, that schematic is the authoritative content and the specification below states how to render it formally.

#	Title	Purpose	Recommended tool / notation
1	Problem Context Diagram	Show the hospital, its infrastructure systems, and the operator, with no internal ResilienceOS detail — establishes the problem before the solution.	Mermaid flowchart, top-down
2	Overall System Architecture	The Part IV master architecture diagram, components top (telemetry/config) to bottom (3D twin).	draw.io / Mermaid flowchart
3	Digital Twin Architecture	Entities → state → relationships → simulation → predicted future states (Part XI).	Mermaid flowchart
4	Infrastructure Dependency Graph	Grid → Transformer → Emergency Bus → Generator/UPS → Services, with relationship labels (SUPPLIES/FEEDS/POWERS/BACKS_UP).	Neo4j Bloom export or Mermaid graph
5	Three-Layer Hospital Model	Infrastructure / Services / Criticality layers stacked (Part VIII of the source spec, reflected in Sections 8–9).	PlantUML layered diagram
6	IoT Data Flow	Simulator → Backend → State Engine → Storage (Part VII).	Mermaid flowchart
7	Real-Time WebSocket Flow	Simulator → FastAPI → WebSocket → Dashboard → 3D Twin (Section 75).	Mermaid sequence diagram
8	Failure Propagation Flow	The ten-step Cascade Engine process (Part IX/XVII).	Mermaid flowchart
9	Main Power Failure Sequence	Transformer → Bus → Generator → UPS → Battery → Service Risk (Section 89).	Mermaid sequence diagram
10	Cascade Timeline	T+0 / T+5 / T+10 / T+20 with changing metrics (Section 21).	Mermaid Gantt or timeline diagram
11	What-If Simulation Workflow	Current State → Intervention → Transition → Impact → Evaluation (Part X).	Mermaid flowchart
12	Response Evaluation Architecture	Simulation outputs → objective function → ranked (not auto-applied) comparison (Part X/XLV).	Mermaid flowchart
13	Resilience Index Model	C, A, B, T, U sub-scores combining into R (Section 108).	PlantUML or a simple weighted-sum diagram

#	Title	Purpose	Recommended tool / notation
14	Hybrid Intelligence Architecture	Graph/Rules + ML + Simulation + Evaluation + NL Explanation, each answering a distinct question (Part XIV).	Mermaid flowchart, four parallel lanes
15	ML Pipeline	Telemetry → Feature extraction → Anomaly scoring → Risk combination (Part VIII).	Mermaid flowchart
16	Neo4j Graph Schema	Node labels, relationship types, and their properties (Part VI/XXXVI).	Neo4j Arrows.app export
17	PostgreSQL ER Diagram	Tables from Part XXV with keys and relationships.	draw.io ER notation
18	API Architecture	REST + WebSocket endpoints grouped by resource (Part XXXVII).	draw.io / OpenAPI-generated diagram
19	Component Diagram	Frontend / Backend / Graph / ML / Simulation as deployable components with interfaces.	UML component diagram (PlantUML)
20	Deployment Architecture	Browser → React → FastAPI → PostgreSQL/Neo4j/ML/Simulation, containerized (Part LXII).	draw.io deployment diagram
21	User Journey	The ten-step operator journey (Section 49).	Mermaid journey diagram
22	Dashboard Information Architecture	Current State → Incident → Infrastructure → Services → Cascade → Response → Impact (Section 29/113).	Simple box hierarchy diagram
23	3D Twin Interaction Flow	Camera/scene states in response to incident/state changes (Part XXVII).	Storyboard-style flow
24	Recovery Flow	AT RISK/FAILED → RECOVERING → NORMAL, with rate determined by strategy (Section 96).	Mermaid state diagram
25	End-to-End System Flow	The full observe → understand → predict → simulate → compare → explain → decide → recover loop.	Mermaid flowchart, matches Part I Section 6

Appendix — Configuration, Example Telemetry, Graph, and API Payloads

175. Configuration Parameters

The hospital model is defined declaratively. A representative asset entry:

```
{
  "asset_id": "GEN-01",
  "type": "generator",
  "capacity_kw": "<configurable prototype parameter>",
  "criticality": "<configurable prototype parameter>",
  "backup_priority": 1,
  "activation_delay_seconds": "<configurable prototype parameter>",
  "threshold_fraction": "<configurable prototype parameter>"
}
```

A representative service entry:

```
{
  "service_id": "ICU-01",
  "type": "service",
  "name": "Intensive Care Unit",
  "beds": 6,
  "criticality": 5,
  "dependencies": [
    {"asset_id": "EMERGENCY-BUS-01", "relation": "POWERS", "strength": 1.0},
    {"asset_id": "HVAC-PLANT-01", "relation": "COOLS", "strength": 0.6},
    {"asset_id": "MEDGAS-01", "relation": "PROVIDES_GAS", "strength": 0.8},
    {"asset_id": "WATER-PUMP-01", "relation": "PROVIDES_WATER", "strength": 0.3}
  ]
}
```

Values shown as <configurable prototype parameter> are deliberately left as placeholders here rather than given invented numbers; the actual seed configuration file assigns concrete values chosen for demo clarity and documents them as such where they are set.

176. Example Telemetry

Illustrative example only — not a captured system output:

```
{
  "asset_id": "GEN-01",
  "timestamp": "T+00:05:00",
  "state": "STARTING",
  "load_kw": 0,
  "target_load_kw": 180
}
{
  "asset_id": "BATTERY-01",
  "timestamp": "T+00:05:00",
  "state": "DISCHARGING",
  "reserve_pct": 51,
  "depletion_rate_pct_per_min": 1.2
}
```

177. Example Graph

Illustrative fragment of the dependency graph around the hero scenario:

```
(Grid)-[:SUPPLIES]->(Transformer:T1)
(T1)-[:FEEDS]->(EmergencyBus:B1)
(EmergencyBus:B1)-[:POWERS]->(ICU)
(Generator:GEN-01)-[:BACKS_UP]->(EmergencyBus:B1)
(UPS:UPS-01)-[:BACKS_UP {priority:1}]->(EmergencyBus:B1)
(HVACPlant:HVAC-01)-[:COOLS]->(ICU)
(MedicalGasSystem:MEDGAS-01)-[:PROVIDES_GAS]->(ICU)
```

178. Example API Payloads

See Part XXIV, Section 124 for the `/failures/inject` example; below is an illustrative `/simulation/run` exchange.

```
Request:
{
  "incident_id": "inc_0007",
  "strategies": ["A", "B", "C", "D", "E", "F"],
  "horizon_minutes": 30
}

Response (200):
{
  "incident_id": "inc_0007",
  "results": {
    "A": {"continuity": 0.61, "backup_margin": 0.20, "recovery_min": null, "resilience_index":
43},
    "C": {"continuity": 0.94, "backup_margin": 0.55, "recovery_min": 22, "resilience_index": 91}
    /* ... remaining strategies omitted for brevity ... */
  }
}
```

These numeric values are illustrative examples used to show the payload shape, not measured system output.

179. Example Scenario

The seed "Main Power Failure" scenario used throughout the demo: origin = TRANSFORMER-01, severity = high, duration = 20 minutes (configurable), with the baseline hospital configuration described in Part V. This is the scenario referenced by the hero demo (Part XVIII) and the scenario tests (Part XIV, Section 134).

References

Sources actually consulted for this document. Numbers correspond to inline citation markers used throughout.

1. Rinaldi, S. M., Peerenboom, J. P., & Kelly, T. K. (2001). Identifying, Understanding, and Analyzing Critical Infrastructure Interdependencies. *IEEE Control Systems Magazine*, 21(6), 11–25.
2. National Fire Protection Association. NFPA 99: Health Care Facilities Code (Essential Electrical System requirements, Chapter 6) and NFPA 110: Standard for Emergency and Standby Power Systems (generator start-time and testing requirements). Referenced via secondary industry summaries of these standards; consult the current published NFPA 99/110/101/70 editions directly for authoritative requirements.
3. Theoharidou, M., Kotzanikolaou, P., & Gritzalis, D. (2010). A multi-layer criticality assessment methodology based on interdependencies. *Computers & Security*, 29(6), 643–658. (Multi-order dependency-graph extensions of the Rinaldi et al. framework.)
4. National Institute of Standards and Technology. NISTIR 8356 (draft): Considerations for Digital Twin Technology and Emerging Standards. NIST, 2021.
5. Wikipedia contributors. "Digital twin." Wikipedia, The Free Encyclopedia — summary of the distinction between continuously-synchronized digital twins and static/offline simulation models presented as digital twins.
6. Balasubramanyam, A., Ramesh, R., Sudheer, R., & Honnavalli, P. B. (2024). Revolutionizing Healthcare: A Review Unveiling the Transformative Power of Digital Twins. *IEEE Access*, 12, 69652–69676.
7. Xames, M., & Topcu, T. (2024). A Systematic Literature Review of Digital Twin Research for Healthcare Systems: Research Trends, Gaps, and Realization Challenges. *IEEE Access*, 12, 4099+. Mukherjee, A., Moretti, N., Chan, Y.-C., Yilmaz, G., Sasidharan, M., Merino, J., Rosun, Z., Carr, C., McFarlane, D., & Parlikad, A. K. (2023). Healthcare Process and Variable Mapping For Identifying Digitalisation Opportunities and Building a Hospital Digital Twin. *Proceedings of the 2023 European Conference on Computing in Construction (EC3)*.
8. Liu, F. T., Ting, K. M., & Zhou, Z.-H. (2008). Isolation Forest. *Proceedings of the 2008 Eighth IEEE International Conference on Data Mining (ICDM)*, 413–422. DOI: 10.1109/ICDM.2008.17.

No paper, DOI, or dataset not listed above is cited elsewhere in this document. Where a claim in the body text has no citation, it is a project design decision, prototype assumption, or illustrative example per the Claim Audit, not an external factual claim requiring a source.

Appendix — Glossary

Term	Definition as used in this document
Cascade	The propagation of a state change from a failed/degraded node through the dependency graph to downstream nodes.
Criticality	A configurable 1–5 score reflecting how much a service's continuity is weighted in resilience and evaluation calculations.
Dependency graph	The Neo4j property graph of infrastructure and service nodes and their typed relationships.
Digital twin (as used here)	A prototype digital twin of a simulated hospital: a continuously state-driven virtual representation capable of simulating future states, per Part XI.
Resilience Index	A project-defined 0–100 composite score summarizing modeled infrastructure/service health (Part XI).
Service risk	A modeled, infrastructure-derived risk indicator for a hospital service — never a clinical or patient-level probability.
State machine	The NORMAL / DEGRADED / AT RISK / FAILED transition model applied to infrastructure nodes (Part XXXVIII).
Strategy (A–F)	One of six defined response interventions the What-If Simulator can apply and compare (Part X).
Threshold	A configured value below which a node's available capacity is considered to affect its own or a downstream node's state.
What-if simulation	Running a hypothetical intervention forward through the cascade/state logic to produce comparable outcome metrics.

Final ResilienceOS Architect Review

A. What is strongest about the project?

The separation between infrastructure state and service state (Part XVIII/IX), and the fact that explanations are a direct byproduct of the graph traversal and simulation computation rather than a separately generated justification. This combination is what lets the system show its work, which is the single most defensible property it has under judge questioning.

B. What is weakest?

The Resilience Index and its weights are a design choice with no external validation, and the six response strategies, while well-motivated, are a fixed, hand-designed set rather than a general intervention search. Both are honest limitations, not hidden ones, but both are easy targets for a skeptical judge.

C. What could a judge attack?

The absence of any real dataset (ML and simulation validation), the arbitrariness of specific configured numbers (capacities, thresholds, criticality scores), and the risk that the 3D twin is mistaken for the product itself rather than the visualization of the actual product (the graph/simulation/evaluation stack).

D. What is technically risky?

End-to-end integration across five different technologies (React, FastAPI, Neo4j, PostgreSQL, Three.js) within a compressed timeline is the single largest technical risk — not the difficulty of any one component, but the number of interfaces between them that must all work simultaneously for the demo loop to run.

E. What is likely to consume the most development time?

Getting the cascade engine's threshold/capacity logic to produce a cascade that is both technically defensible and visually clear in the demo — the two goals sometimes pull in different directions (a too-simple cascade looks unconvincing; a too-complex one is hard to explain in the time available).

F. What should the team build first?

The graph, the stateful simulator, and the cascade engine (Phases 1–5, Part XVII) — the Core MVP loop depends entirely on this triad working correctly before dashboard polish or 3D fidelity add any real value.

G. What should the team avoid building?

A general-purpose optimization/search engine for interventions, a multi-agent LLM architecture, SHAP/LIME attribution over a formula that is already transparent, and any additional infrastructure system beyond electrical/backup/water/HVAC/medical-gas unless it creates a genuinely new dependency the demo needs (Section 67, Anti-Buzzword Audit).

H. What features are optional?

The incident timeline scrubber (FR-17), full anomaly-detection ML (a statistical fallback is acceptable), and time-to-threshold estimation (FR-11) are all Valuable-if-time-permits rather than Essential — the demo narrative survives without any one of them, though all three strengthen it.

I. What must work for the demo?

Failure injection, cascade propagation with a visible dependency chain, at least three of the six what-if strategies with comparable metrics, a Resilience Index that visibly changes across states, one working "Why?" explanation, and a visible recovery animation — this is the non-negotiable Core MVP (Part XX, Section 91).

J. What claims should never be made?

- That any number in the system is a validated clinical, engineering, or experimental result.
- That the system predicts patient outcomes or risk.
- That the system is clinically validated, certified, or compliant with any specific jurisdiction's code.
- That the system can or does control real hospital infrastructure.
- That the project is the first, only, or best system of its kind.

K. What research areas should the team continue reading?

- Formal resilience-engineering metrics (beyond the project-defined Resilience Index) for eventual comparison/calibration.
- Graph-based cascading-failure simulation methods in the broader critical-infrastructure-protection literature.
- Healthcare-specific digital-twin case studies (e.g., the Cambridge histopathology case study) for patterns applicable to infrastructure rather than clinical processes.
- NFPA 99/110/101/70 (or the equivalent applicable code in the team's jurisdiction) if the project moves toward any real facility engagement.

L. What would be the logical next version after the hackathon?

A version with a wider synthetic scenario library for proper ML evaluation, a slightly larger modeled hospital (more assets and at least one additional infrastructure system), a first attempt at a read-only integration with a real (non-production) BMS data feed for validation purposes only, and a first formal write-up suitable for a workshop-level paper on the integration approach itself — not on any claimed real-world result, since none would yet exist.

Final Master Project Statement

ResilienceOS is a living digital model of critical hospital infrastructure that understands system dependencies, predicts how failures may cascade, simulates possible responses, and helps operators protect critical hospital services before disruption spreads.

The core idea is simple: normal monitoring tells you something failed. ResilienceOS helps you understand what that failure could affect next and explore what to do about it before acting.